

Hadamard Product Arguments and Their Applications

Kyeongtae Lee¹[0009–0007–7969–9289], Donghwan Oh¹[0009–0001–2728–9460],
Hankyung Ko²[0000–0001–9430–1768], Jihye Kim³[0000–0003–2953–7883], and
Hyunok Oh¹[0000–0002–9044–7441]

¹ Department of Information Systems, Hanyang University, Seoul 04763, South Korea
`{rsias9049,donghwanoh,hoh}@hanyang.ac.kr`

² Radius, Seoul 06232, South Korea
`hankyungko@hanyang.ac.kr`

³ Electronics and Information System Engineering Major, Kookmin University, Seoul
02707, South Korea
`jihyek@kookmin.ac.kr`

Abstract. The Hadamard product (also known as element-wise multiplication) is a fundamental operation in linear algebra, performed by multiplying corresponding elements of two matrices with the same dimensions. This operation plays a crucial role in various fields, including cryptography, where it enables efficient and parallelizable computations on large datasets—particularly in the design of cryptographic protocols such as zero-knowledge proofs.

In this paper, we propose a transparent and efficient method for proving the Hadamard product between vectors that are independently committed in the groups $\mathbb{G}_1$ and $\mathbb{G}_2$ under a pairing operation $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. For a vector of length n , the prover has a complexity of $\mathcal{O}_\lambda(n)$, while the proof size is $\mathcal{O}_\lambda(\log n)$. The verifier operates with a complexity of $\mathcal{O}_\lambda(\log n)$, which includes $\mathcal{O}(\log n)$ operations in $\mathbb{G}_T$ and only $\mathcal{O}(1)$ pairing operations, making verification highly efficient. We prove the security of our scheme under the Symmetric External Diffie-Hellman (SXDH) assumption.

Furthermore, we propose an aggregator for Groth16 (EUROCRYPT 2016) zk-SNARKs and a proof aggregation technique for the general case of the KZG polynomial commitment scheme (ASIACRYPT 2010), where all crs are distinct. Both applications do not require an additional trusted setup, support logarithmic-sized aggregated proofs, and significantly reduce the verifier’s pairing operations to $\mathcal{O}(1)$.

Keywords: Zero-knowledge proof, Pairing-based cryptography, SNARK, Privacy

1 Introduction

The role of inner product and Hadamard product, also known as entry-wise products, is fundamental in modern cryptography. These operations are crucial for

supporting a wide array of cryptographic primitives, including zero-knowledge proofs and commitment schemes [5,27,7,8].

In Groth’s work [16], an extensive framework for zero-knowledge arguments is introduced, primarily emphasizing the reduction of Hadamard products to inner product relations. This scheme employs a modified form of Pedersen commitments [25] and integrates randomization and batch-verification techniques to streamline diverse linear algebra relations into inner product relations. The randomization technique is succinctly explained as follows: consider a randomly chosen scalar $r \in \mathbb{F}$. Introduce a binary operator $*$, which maps from $\mathbb{F}^n \times \mathbb{F}^n$ to $\mathbb{F}$ and is defined as $\mathbf{a} * \mathbf{b} = \mathbf{a}(\mathbf{b} \circ \mathbf{r})^T$, where $\mathbf{r} = (1, r, r^2, \dots, r^{n-1})$. An interesting property emerges: if $\mathbf{a} * \mathbf{b} = \mathbf{c} * \mathbf{d}$, it implies $\mathbf{a} \circ \mathbf{b} = \mathbf{c} \circ \mathbf{d}$, with an error bound of at most $(n-1)/|\mathbb{F}|$. Incorporating this randomization technique enables the demonstration of Hadamard Product Arguments (HPA) from Inner Product Arguments (IPA), expressed as $\langle \mathbf{a}_r, \mathbf{b} \rangle = \langle \mathbf{c}_r, \mathbf{d} \rangle$. Consequently, leveraging efficient IPA, such as [7,8] allows the establishment of efficient HPA protocols.

The Bulletproofs [7] achieves transparency but requires a linear verification cost for the verifier. Conversely, Bünz et al. [8], achieves sublinear verification times but necessitates a trusted setup. Both approaches face limitations due to the verifier computing the commitment keys: unstructured keys, while transparent, impose a linear computation cost on the verifier, whereas structured keys reduce the verifier’s computation on commitment keys but require an additional trusted setup.

Algorithm 1 Dory-Reduce [22]

Precompute : $\chi = \langle \mathbf{ck}_1, \mathbf{ck}_2 \rangle$,
 $\Delta_{1L} = \langle \mathbf{ck}_{1L}, \mathbf{ck}'_2 \rangle$, $\Delta_{1R} = \langle \mathbf{ck}_{1R}, \mathbf{ck}'_2 \rangle$,
 $\Delta_{2L} = \langle \mathbf{ck}'_1, \mathbf{ck}_{2L} \rangle$, $\Delta_{2R} = \langle \mathbf{ck}'_1, \mathbf{ck}_{2R} \rangle$
Input : $(C, D_1, D_2 ; \mathbf{a}, \mathbf{b}) \in \mathcal{R}_{2^m, \mathbf{ck}_1, \mathbf{ck}_2}$
1: $\mathcal{P} \rightarrow \mathcal{V} : D_{1L} = \langle \mathbf{a}_L, \mathbf{ck}'_2 \rangle$, $D_{1R} = \langle \mathbf{a}_R, \mathbf{ck}'_2 \rangle$,
 $D_{2L} = \langle \mathbf{ck}'_1, \mathbf{b}_L \rangle$, $D_{2R} = \langle \mathbf{ck}'_1, \mathbf{b}_R \rangle$
2: $\mathcal{V} \rightarrow \mathcal{P} : \beta \leftarrow \mathbb{F}$
3: $\mathcal{P} : \mathbf{a}^* \leftarrow \mathbf{a} + \beta \mathbf{ck}_1$, $\mathbf{b}^* \leftarrow \mathbf{b} + \beta^{-1} \mathbf{ck}_2$
4: $\mathcal{P} \rightarrow \mathcal{V} : C_+ = \langle \mathbf{a}_L, \mathbf{b}_R \rangle$, $C_- = \langle \mathbf{a}_R, \mathbf{b}_L \rangle$
5: $\mathcal{V} \rightarrow \mathcal{P} : \alpha \leftarrow \mathbb{F}$
6: $\mathcal{P} : \mathbf{a}' \leftarrow \alpha \mathbf{a}^*_L + \mathbf{a}^*_R$, $\mathbf{b}' \leftarrow \alpha^{-1} \mathbf{b}^*_L + \mathbf{b}^*_R$
7: $\mathcal{V} : C' \leftarrow C + \chi + \beta D_2 + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_-$
 $D'_1 \leftarrow \alpha D_{1L} + D_{1R} + \alpha \beta \Delta_{1L} + \beta \Delta_{1R}$
 $D'_2 \leftarrow \alpha^{-1} D_{2L} + D_{2R} + \alpha^{-1} \beta^{-1} \Delta_{2L} + \beta^{-1} \Delta_{2R}$
Output : $(C', D'_1, D'_2 ; \mathbf{a}', \mathbf{b}') \in \mathcal{R}_{2^{m-1}, \mathbf{ck}'_1, \mathbf{ck}'_2}$

Dory [22], a significant advancement towards transparent arguments, presents an efficient IPA achieving sublinear verification time. Its efficiency leverages Type III pairings and relies on two main techniques. First, a preprocessing phase computes commitments (Δ) to the recursive commitment keys ($\mathbf{ck}$), allowing

the verifier to work with these logarithmic-sized values instead of the full keys. Second, it delegates the core folding computations to the prover, while the verifier homomorphically computes the commitments for the next round (D'_1, D'_2) using the preprocessed Δ values and challenges, ensuring the prover's computations are consistent. To achieve constant-time verification updates in its logarithmic variant, Dory uses a random challenge β to create combined vectors $(\mathbf{a}^* = \mathbf{a} + \beta \mathbf{c}\mathbf{k}_1, \mathbf{b}^* = \mathbf{b} + \beta^{-1} \mathbf{c}\mathbf{k}_2)$ by merging witness and key components before applying the folding step with another challenge α to produce the folded vectors $(\mathbf{a}', \mathbf{b}')$.

While Dory provides a powerful IPA framework, applying Groth's randomization technique [16] to construct a HPA – which involves proving an inner product like $\langle \mathbf{a}, \mathbf{b} \circ \mathbf{r} \rangle$ – directly onto Dory's logarithmic structure faces a fundamental challenge. Groth's method requires replacing one vector (say, $\mathbf{b}$) with its randomized version $\mathbf{b} \circ \mathbf{r}$. If we attempt this in Dory's logarithmic protocol, the combined vector becomes $\mathbf{b}^* = (\mathbf{b} \circ \mathbf{r}) + \beta^{-1} \mathbf{c}\mathbf{k}_2$. Subsequently, the folded vector $\mathbf{b}'$ will contain terms derived from both $\mathbf{b} \circ \mathbf{r}$ and $\mathbf{c}\mathbf{k}_2$, mixed by α and β .

The issue arises during verification. The verifier computes the commitment for the next round, D'_2 (corresponding to the folded vector $\mathbf{b}'$), homomorphically using the challenges α, β , and the preprocessed commitments Δ , which relate to the original, non-randomized keys $\mathbf{c}\mathbf{k}$. This homomorphic computation relies intrinsically on the algebraic structure defined by the original folding process described in [22]. When $\mathbf{b} \circ \mathbf{r}$ is introduced, the randomization factor $\mathbf{r}$ becomes entangled with the commitment key $\mathbf{c}\mathbf{k}$ and the challenges α, β within the folded vector $\mathbf{b}'$. The verifier, possessing only the preprocessed commitments Δ for the original keys, lacks the necessary information to homomorphically compute or verify the commitment D'_2 corresponding to the correctly folded, randomized structure. Essentially, the verifier cannot check if the prover correctly computed $\mathbf{b} \circ \mathbf{r}$ and folded it according to the protocol rules using only the preprocessed data. The mechanism for verifying the delegated folding computation breaks down.

This limitation highlights the difficulty of applying standard randomization techniques when commitment key computations are delegated and combined with witnesses using challenges like β before the folding step itself. To address this challenge and enable a direct, transparent HPA with logarithmic verification, we introduce a novel *elimination folding* technique. This technique is specifically designed to handle the randomization factor $\mathbf{r}$ effectively. It separates the folding of the randomized witness from the commitment key folding, allowing the $\mathbf{r}$ terms to be systematically eliminated during the recursive folding process while maintaining the verifier's ability to efficiently check the protocol's execution.

To provide further detail, given witness vectors $\mathbf{a}$ and $\mathbf{a}_{\mathbf{r}}$, along with a random challenge $r \leftarrow \mathbb{F}$, the verifier aims to verify that $\mathbf{a}_{\mathbf{r}}$ and $(1, r, r^2, \dots, r^{n-1}) \circ \mathbf{a}$ are equal. To achieve this, we employ an iterative halving process for the degree of r through a folding technique, ultimately deriving scalar a and a_r . Verification of their equality ensures that $\mathbf{a}_{\mathbf{r}}$ is indeed equal to $\mathbf{r} \circ \mathbf{a}$ (Further details about the elimination folding technique, refer to section 1.3).

1.1 Our Contributions

Hadamard Product Arguments. We propose an efficient Hadamard product argument, named Π_{HP} . The Π_{HP} maintains transparency and ensures a verification time of $\mathcal{O}_\lambda(\log n)$, sublinear proof size, and $\mathcal{O}_\lambda(n)$ proving time. To the best of our knowledge, this work represents the first sublinear-size HPA with a transparent setup. Moreover, we establish the security of our scheme under the Symmetric External Diffie-Hellman (SXDH) assumption. To achieve this, we propose the elimination folding technique. This novel method enables the effective folding of the vector and the elimination of half of the exponents from the random vector $\mathbf{r}$. Consequently, both the resulting vectors $\mathbf{a}_{\mathbf{r}}$ and $\mathbf{r} \circ \mathbf{a}$ simplify to scalars, ensuring identical values due to the removal of randomness in r . This advancement empowers efficient verification, confirming whether $\mathbf{a}_{\mathbf{r}}$ equals $\mathbf{r} \circ \mathbf{a}$, and stands as a crucial element in the development of our efficient Hadamard product arguments protocol.

Table 1: Comparison of asymptotic performance of Hadamard Product Arguments. [6,30,29] are interactive protocols where the parties have access to (polynomial) oracles. To instantiate oracles, [29] uses Lagrange-based KZG commitment, [6] uses tensor code.

Protocol	Transparency	complexity			Security assumption
		$\mathcal{P}$'s time	Proof size	$\mathcal{V}$'s time	
Ours	Yes	$\mathcal{O}_\lambda(n)$	$\mathcal{O}_\lambda(\log n)$	$\mathcal{O}_\lambda(\log n)$	SXDH
Bulletproofs [7]	Yes	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$	DL
VOPROOF [30] + KZG	No	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$	-
Xie et al. [29] + Lagrange KZG	No	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	-
Bünz et al. [8]	No	$\mathcal{O}_\lambda(n)$	$\mathcal{O}_\lambda(\log n)$	$\mathcal{O}_\lambda(\log n)$	ASDBP, SDH, GGM
Poppins [21]	No	$\mathcal{O}_\lambda(n)$	$\mathcal{O}_\lambda(\log n)$	$\mathcal{O}_\lambda(\log n)$	BSDH, EPKE
Daza et al. [11]	No	$\mathcal{O}_\lambda(n)$	$\mathcal{O}_\lambda(\log n)$	$\mathcal{O}_\lambda(\log n)$	q-A-DLOG
Lipmaa et al. [24]	No	$\mathcal{O}_\lambda(n \log n)$	$\mathcal{O}(1)$	$\mathcal{O}_\lambda(1)$	AGM, PDL

Efficient aggregation of pairing equations using Hadamard product arguments. In this work, we introduce an efficient method for aggregating in the verification of pairing equations using Hadamard product arguments. Our key contributions are as follows:

- We address the most general case where all a_i, b_i, c_i, d_i are distinct. Here, a_i, b_i, c_i , and d_i are elements of the groups involved in the pairing operation. Traditional methods of randomized aggregation (as [20,4]) become inefficient in this setting. We propose a novel approach using Hadamard product arguments, which reduces the number of pairing operations required for verification. This makes the verification process more efficient.
- Our technique can be applied to various cryptographic protocols that rely on pairing equations, such as digital signatures, zero-knowledge proofs, and

commitment schemes. By improving the efficiency of these schemes, our method facilitates faster and more practical implementations.

- We demonstrate that our approach significantly enhances scalability and performance in the verification of pairing equations. By reducing the computational burden on the verifier, our method ensures faster verification processes, which is critical for large-scale cryptographic applications. The reduction in pairing operations leads to substantial improvements in both time and resource efficiency.

Through these contributions, we provide a framework that not only advances the state-of-the-art in proof aggregation but also offers practical benefits for a wide range of cryptographic protocols and applications.

Proof Aggregation. We propose an aggregator for Groth16 [18] pairing-based zero-knowledge Succinct Non-interactive Arguments of Knowledge (zk-SNARKs) and a proof aggregation technique for the most general case of the KZG [20] pairing-based polynomial commitment scheme using our Π_{HP} . Both applications support logarithmic-sized aggregated proof without requiring an additional trusted setup and significantly reduce the verifier’s $\mathcal{O}(n)$ pairing operations to $\mathcal{O}(1)$ pairing operations.

Implementation. To demonstrate the efficiency of our proposed Π_{HP} , we implement and optimize it using batch proving techniques. The evaluation results reveal that Π_{HP} employs a logarithmic verifier, a notable improvement over the linear verifier associated with Bulletproofs [7].

1.2 Applications

This subsection introduces applications where our Hadamard product arguments can be applied. Consider a scheme with a verification equation $e(a, b) = e(c, d)$. Verifying n pairing equations, i.e., $e(a_i, b_i) = e(c_i, d_i)$ for $i \in [n]$, requires the verifier to perform $2n$ pairing operations. However, if in the aforementioned pairing equations, all b_i ($b := b_i$) are the same (as in BLS multi-signature [4]), the prover can aggregate a_i into a single proof using a challenge $r \leftarrow \mathbb{F}$. Consequently, the system of pairing equations reduces to $e(\prod a_i^{r^{i-1}}, b) = \prod_{i=1}^n e(c_i, d_i)^{r^{i-1}}$, thus reducing the verifier’s pairing operations from $2n$ to $n + 1$. Additionally, if all d_i ($d := d_i$) are also the same (as in the case of the same common reference strings (crs) and point for KZG commitment [20]), we obtain the pairing equation $e(\prod a_i^{r^{i-1}}, b) = e(\prod c_i^{r^{i-1}}, d)$. This allows the verifier to verify all n verification equations with just 2 pairing operations.

We consider the most general case where all a_i, b_i, c_i, d_i are distinct. That is, we address the case of having n verification equations of the form $e(a_i, b_i) = e(c_i, d_i)$. In this case, randomized proof aggregation is no longer efficient, but verifier-efficient proof aggregation is achievable using our Hadamard product arguments.

Aggregation for Groth16 Groth16 [18] presents an efficient zk-SNARK based on pairings, where the verifier verifies the equation $e(A, B) = e(C, D) \cdot e(S, H)$ using the proof $\pi = (A, B, C)$ (for detailed explanation, refer to Appendix A). Therefore, to aggregate n instances of Groth16, the verifier needs to verify the pairing equations $e(A_i, B_i) = e(C_i, D_i) \cdot e(S_i, H_i)$ for all $i \in [n]$. We consider each proof as a vector and contemplate their Hadamard product: $\mathbf{A} \circ \mathbf{B} = \mathbf{C} \circ \mathbf{D} + \mathbf{S} \circ \mathbf{H}$. Our approach reduces the verifier's required pairings from $\mathcal{O}(n)$ to $\mathcal{O}(1)$, efficiently improving the verification time for aggregation (for detailed explanation, refer to Section 6). Notably, this protocol enables the aggregation of proofs generated using different verification keys, offering a flexible approach particularly relevant for scenarios like prover markets where proofs may originate from diverse, independent source[28]. Furthermore, while we demonstrate this application using Groth16 as a prominent example, the underlying aggregation technique based on our HPA is generic and can be similarly applied to other recent pairing-based SNARKs, such as those presented in [12].

KZG commitment Aggregation for distinct crs KZG commitment [20] is a widely used polynomial commitment. Its verification involves pairing equations, where the verifier checks $e(C - g^a, h) = e(\pi, h^{z-s})$ (for detailed explanation, refer to Appendix B). As mentioned earlier, in the most general case where $\mathbf{crs}$, polynomials, and points are all different, the above equation manifests into n pairing equations as follows: for all $i \in [n]$, $e(C_i - g^{a_i}, h) = e(\pi_i, h^{z_i-s_i})$. Similarly, we can consider this as vectors and apply Hadamard product arguments: $\mathbf{C} \circ \mathbf{h} = \boldsymbol{\pi} \circ \mathbf{H}$ (for detailed explanation, refer to Section 7).

1.3 Technical Overview

Achieving a transparent HPA with sublinear verification requires overcoming the limitations identified in prior works, such as the linear verification cost of transparent schemes [7] or the trusted setup requirement of sublinear ones [8], as well as the challenges in applying standard randomization techniques [16] to efficient IPA like Dory [22].

Here, we provide an overview of our HPA and its core technique, the elimination folding algorithm, designed to address these issues.

Given two vectors $\mathbf{a}, \mathbf{c} \in \mathbb{G}_1^n$ and $\mathbf{b}, \mathbf{d} \in \mathbb{G}_2^n$, the Hadamard (pairing) product relation is defined as:

$$\mathbf{a} \circ \mathbf{b} = \mathbf{c} \circ \mathbf{d} \in \mathbb{G}_T^n.$$

To efficiently verify this relation, we employ a polynomial-based transformation that reduces the problem to checking a single equation involving inner pairing products:

$$X = \sum_{i=1}^n r^{i-1} e(a_i, b_i), \quad X' = \sum_{i=1}^n r^{i-1} e(c_i, d_i),$$

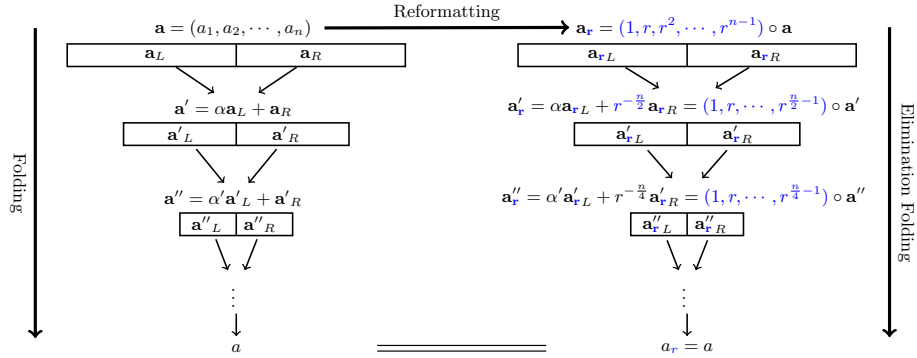
where r is a randomly chosen challenge. By the Schwartz-Zippel lemma, verifying $X = X'$ with high probability guarantees the correctness of the Hadamard product relation.

Elimination Folding Algorithm. A key challenge in efficiently proving the Hadamard product relation is verifying that the vectors $\mathbf{a}_r = \mathbf{r} \circ \mathbf{a}$ and $\mathbf{c}_r = \mathbf{r} \circ \mathbf{c}$ were computed correctly. To address this, we introduce the *elimination folding algorithm*, which progressively reduces the vector length while eliminating the challenge r . This algorithm recursively halves the vector length while maintaining correctness. Given a vector $\mathbf{a} \in \mathbb{G}_1^n$, we iteratively apply the transformation:

$$\mathbf{a}' = \alpha \mathbf{a}_L + \mathbf{a}_R, \quad \mathbf{a}'_r = \alpha \mathbf{a}_{rL} + r^{-n/2} \mathbf{a}_{rR},$$

where α is a verifier-supplied challenge, and subscripts L and R denote the left and right halves of the vector. Since $\mathbf{a}'_r$ has half the length of $\mathbf{a}_r$ and its degree in r is also halved, this process continues for $\log n$ rounds until a single element remains. At this point, all occurrences of r in $\mathbf{a}_r$ have been eliminated, ultimately ensuring that the reduced elements a and a_r are equal. A similar process is applied to $\mathbf{c}_r$. Furthermore, applying the same folding strategy to $\mathbf{b}$, $\mathbf{c}$, and $\mathbf{d}$ allows us to obtain single elements b, c, d .

Fig. 1: Elimination Folding Process. The vectors are folded, reducing the length and eliminating the random challenge r .



Final Pairing Check. After the recursive elimination folding, the verifier is left with a single equation to verify:

$$C = X, \quad D_1 = D_3,$$

where $C = e(a, b)$, $X = e(a_r, b)$, and D_1 and D_3 are commitments to the folded vectors a and a_r . This final pairing check ensures that all prior folding steps were executed correctly and completes the verification process with only a constant number of pairing operations.

Hadamard Product Argument. Our Hadamard product argument achieves logarithmic verification complexity while maintaining linear prover efficiency.

The overall structure of our Hadamard product argument consists of three key components:

- **Reformatting** (Section 3.1): The prover computes $\mathbf{a}_r$ and $\mathbf{c}_r$ and commits to them.
- **Elimination Folding** (Section 3.2): Recursive folding reduces vector length and removes the challenge r .
- **Final Check** (Section 3.3): The verifier performs a pairing check with a single equation.

For further details, refer to Section 3.

1.4 Related Works

Hadamard Product Arguments. Groth introduces the first transparent argument system for various linear algebraic relations, including the Hadamard product, in [16]. The proving time is $\mathcal{O}(n^2)$, and the verification time is $\mathcal{O}(n)$ with $\mathcal{O}(n)$ communication, where n denotes the vector length. Bayer and Groth extend this argument system to multi-Hadamard product arguments in [2], achieving efficiency of $\mathcal{O}(mn)$ for the prover, $\mathcal{O}(n + m)$ for the verifier, and $\mathcal{O}(n + m)$ for communication cost, where m vectors have a length of n .

Another approach to constructing efficient Hadamard product arguments is based on a trusted setup, which generates a **crs** [17,23,13,9,21]. Groth in [17] presents Hadamard product arguments with $\Theta(n^2)$ proving time, $\Theta(1)$ verifying time, and communication cost, but with a **crs** size of $\Theta(n^2)$. Subsequently, [23] improves the **crs** size to $\Theta(n)$ without sacrificing other performance metrics, and [13] further enhances the efficiency of proving time to $\Theta(n \log n)$. In LegoSNARKs [9], Hadamard product arguments with a Commit and Prove paradigm (which combines two relations with shared inputs into one) are presented. Their Hadamard product argument has $\mathcal{O}(n)$ proving time and $\mathcal{O}(\log n)$ verifying time. In Poppins [21], a multi-Hadamard product argument is introduced, improving upon the one presented by Bayer [2]. Poppins’s variant of the multi-Hadamard product argument outsources the verifier’s final check by leveraging the inner product argument of [8], achieving $\mathcal{O}(nm^2)$ prover complexity and $\mathcal{O}(\log n + m)$ verifier complexity. While gaining benefits from verification efficiency, the precondition for a trusted setup arises, which was not needed in Bayer and Groth’s original multi-Hadamard product argument [2].

[29,30] employs Interactive Oracle Proofs (IOP) approaches into the Hadamard product relation. [29] constructs a Hadamard product protocol from an inner product protocol, achieving $\mathcal{O}(n)$ proving time and $\mathcal{O}(\log n)$ verifying time. However, this approach inherits the trusted setup of the underlying KZG polynomial commitment scheme. [30] introduces vector oracle proofs for the Hadamard relation and compiles them into a polynomial IOP, achieving prover performance of $\mathcal{O}(n \log n)$ and verifier performance of $\mathcal{O}(1)$. However, this approach also requires a trusted setup for the KZG polynomial commitment scheme.

Groth16 Proofs Aggregation. Bünz et al. [8] and Snarkpack [15] introduce proof aggregation schemes enabling the aggregation of n Groth16 [18] zk-SNARKs proofs with $\mathcal{O}(\log n)$ proof size and verifier time. Both approaches, by Bünz et al. and Snarkpack, necessitate a specific trusted setup to construct the structured reference string required for verifying such aggregated proofs. They share the conceptual premise that pairing-based SNARKs like Groth16 can be proven and verified solely using algebraic operations [8]. This implies that they can consolidate n Groth16 verification equations into one randomized verification equation using inner product arguments.

KZG commitments Aggregation Plonk [14] and Boneh et al. [3] delve into aggregation techniques using KZG commitments. Plonk introduces a method to aggregate KZG commitments for the same point across different polynomials. This enables efficient combination when multiple polynomials have evaluation values at the same point. In this case, the proof size scales with the number of polynomials, yet the verifier only requires 2 pairing operations. Conversely, Boneh et al. investigate the aggregation of KZG commitments for different points across different polynomials. This presents a technique to efficiently combine commitments when each polynomial has evaluation values at different points. Similar to Plonk, the verifier utilizes only 2 pairing operations, and the proof size is reduced to a constant.

Building upon these foundational works, we extend the aggregation capabilities to handle the most general case where each polynomial has evaluation values at different points, and additionally, each polynomial may be associated with different crs. This method allows the verifier to still require only 2 pairing operations while supporting a logarithmic proof size relative to the number of polynomials and points involved.

2 Preliminaries

2.1 Notations

Let $[n] = \{1, 2, \dots, n\}$. We denote vectors using lowercase boldface letters, such as $\mathbf{a}$, and their components are expressed as $\mathbf{a} = (a_1, a_2, \dots, a_n)$. The inner product of vectors $\mathbf{a}$ and $\mathbf{b}$ is denoted as $\langle \mathbf{a}, \mathbf{b} \rangle$, and their Hadamard product is represented as $\mathbf{a} \circ \mathbf{b}$. Concatenation of vectors $\mathbf{a}$ and $\mathbf{b}$ is denoted as $(\mathbf{a} \parallel \mathbf{b})$. For a vector $\mathbf{a}$ of even length, we use $\mathbf{a}_L$ and $\mathbf{a}_R$ to represent its left and right halves, respectively. If S is a set, “ $x \leftarrow S$ ” indicates sampling x uniformly at random from S . We utilize Type III pairing $(\mathbb{F}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g, h)$ with the following properties:

1. $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order p with generators $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$,
2. The pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a non-degenerate bilinear map,
3. The value $e(g, h)$ generates $\mathbb{G}_T$.

In Type III pairings, we use the following inner pairing product operation [8]:

$$\langle \cdot, \cdot \rangle : \mathbb{G}_1^n \times \mathbb{G}_2^n \rightarrow \mathbb{G}_T, \quad \langle \mathbf{a}, \mathbf{b} \rangle = \prod_{i=0}^n e(a_i, b_i)$$

2.2 Assumptions

Definition 1 (Symmetric external Diffie-Hellman (SXDH) [1]). The *Symmetric XDH* (or *SXDH*) assumption holds for a Type III pairing $(\mathbb{F}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g, h)$ if the decisional Diffie-Hellman (DDH) assumption holds for $(\mathbb{F}_p, \mathbb{G}_1, g)$ and $(\mathbb{F}_p, \mathbb{G}_2, h)$.

Lemma 1. Let $(\mathbb{F}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g, h)$ be a Type III pairing satisfying SXDH assumption and $n = \text{poly}(\lambda)$. Then, given vector $\mathbf{b} \leftarrow \mathbb{G}_{i \in \{1,2,T\}}^n$ there is no non-uniform polynomial-time adversary $\mathcal{A}$ which can compute a non-trivial $\mathbf{a} \in \mathbb{F}^n$ such that $\langle \mathbf{a}, \mathbf{b} \rangle = 0$.

Lemma 2. Let $(\mathbb{F}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g, h)$ be a Type III pairing satisfying SXDH assumption. Then, given $a_1, a_2 \leftarrow \mathbb{G}_1$ there is no non-uniform polynomial-time adversary $\mathcal{A}$ which can compute non-trivial $b_1, b_2 \in \mathbb{G}_2$ such that $e(a_1, b_1) + e(a_2, b_2) = 0$. In general, for a given vector $\mathbf{a} \leftarrow \mathbb{G}_1^n$, there is no adversary which can compute non-trivial vector $\mathbf{b} \in \mathbb{G}_2^n$ such that $\langle \mathbf{a}, \mathbf{b} \rangle = 0$. Similarly, there exists no non-uniform polynomial-time adversary which can compute non-trivial vector $\mathbf{a} \in \mathbb{G}_1^n$ such that $\langle \mathbf{a}, \mathbf{b} \rangle = 0$ for $\mathbf{b} \leftarrow \mathbb{G}_2^n$.

2.3 Commitment scheme

Definition 2 (AFGHO commitment [1]). The AFGHO vector commitment scheme is a structure preserving commitment to group elements. For some message $\mathbf{m} \in \mathbb{G}_1^n$, the triple of PPT algorithms $(\text{KeyGen}, \text{CM}, \text{Open})$ are defined as follows:

- $\text{pp} \leftarrow \text{KeyGen}(1^\lambda) = \{\text{ck} \leftarrow \mathbb{G}_2^n, H_i \leftarrow \mathbb{G}_i\}$
- $(C, o) \leftarrow \text{CM}(\text{pp}; \mathbf{m}) = \{r \leftarrow \mathbb{F}; \langle \mathbf{m}, \text{ck} \rangle + r \cdot e(H_1, H_2), r\}$
- $1/\perp \leftarrow \text{Open}(\text{pp}; C, \mathbf{m}, o) = \{\langle \mathbf{m}, \text{ck} \rangle + o \cdot e(H_1, H_2) \stackrel{?}{=} C\}$

Definition 3 (Computational binding). A triple of three algorithms $(\text{KeyGen}, \text{CM}, \text{Open})$ provides computational binding property if for any PPT adversary $\mathcal{A}$ having knowledge of ck :

$$\Pr \left[\begin{array}{l} \text{Open}(\text{ck}, C, \mathbf{m}, o) \\ \wedge \text{Open}(\text{ck}, C, \mathbf{m}', o') \\ \wedge \mathbf{m} \neq \mathbf{m}' \end{array} \middle| (C, \mathbf{m}, o, \mathbf{m}', o') \leftarrow \mathcal{A}(\text{ck}) \right] \leq \text{negl}(1^\lambda).$$

Definition 4 (Perfect hiding). A triple of three algorithms $(\text{KeyGen}, \text{CM}, \text{Open})$ provides perfect hiding property if for all unbounded adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$:

$$\Pr \left[b = b' \left| \begin{array}{l} \mathbf{ck} \leftarrow \text{KeyGen}(1^\lambda) \\ \wedge (\mathbf{a}_0, \mathbf{a}_1, st) = \mathcal{A}_0(\mathbf{ck}) \\ \wedge b \leftarrow_{\$} \{0, 1\} \\ \wedge (C, o) \leftarrow \text{CM}(\mathbf{ck}, \mathbf{a}_b) \\ \wedge b' \leftarrow \mathcal{A}_1(C, st) \end{array} \right. \right] = \frac{1}{2}.$$

Definition 5 (Doubly homomorphism). A triple of three algorithms $(\text{KeyGen}, \text{CM}, \text{Open})$ provides doubly homomorphism if it satisfies the following conditions:

- The sets $(\mathcal{K}, +)$, $(\mathcal{M}, +)$, and $(\text{Image}(\text{CM}), +)$ form abelian groups.
- For any $\mathbf{ck}, \mathbf{ck}' \in \mathcal{K}$ and $\mathbf{m}, \mathbf{m}' \in \mathcal{M}$, it holds that:
 - $\text{CM}(\mathbf{ck}; \mathbf{m}) + \text{CM}(\mathbf{ck}; \mathbf{m}') = \text{CM}(\mathbf{ck}; \mathbf{m} + \mathbf{m}')$,
 - $\text{CM}(\mathbf{ck}; \mathbf{m}) + \text{CM}(\mathbf{ck}'; \mathbf{m}) = \text{CM}(\mathbf{ck} + \mathbf{ck}'; \mathbf{m})$.

Note that, AFGHO commitment is perfectly hiding, computationally binding, and doubly homomorphic.

2.4 Zero-Knowledge Succinct interactive arguments of knowledge

We follow the presentation in [26,22]. Let $(\mathcal{P}, \mathcal{V})$ be a pair of interactive probabilistic polynomial time (PPT) algorithms. In zero-knowledge succinct arguments of knowledge, the prover $\mathcal{P}$ wants to convince $\mathcal{V}$ that some statement holds and she/he has the key knowledge of why the statement holds without leaking any information of the knowledge.

Definition 6 (Public coin succinct interactive arguments of knowledge). A succinct interactive argument of knowledge scheme for a NP language $\mathcal{R}$ is a protocol between $(\mathcal{P}, \mathcal{V})$ with following properties:

- **Completeness** : An argument is complete if given true statement $x \in \mathcal{R}$, for any witness w and $r \in \{0, 1\}^*$, $\Pr[\langle \mathcal{P}(pp, w), \mathcal{V}(pp, r) \rangle(x) = 1] = 1$.
- **Soundness** : An argument is sound if for $x \notin \mathcal{R}$, any PPT Prover $\mathcal{P}^*$, and for all $r \in \{0, 1\}^*$, $\Pr[\langle \mathcal{P}^*(pp), \mathcal{V}(pp, r) \rangle(x) = 1] \leq \text{negl}(\lambda)$.
- **Knowledge soundness** : An argument is knowledge sound if for any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$ such that $\forall x \in \mathcal{R}, \forall r \in \{0, 1\}^*$, if $\Pr[\langle \mathcal{A}(pp), \mathcal{V}(pp, r) \rangle(x) = 1] \geq \text{negl}(\lambda)$, then $\Pr[\text{Sat}_{\mathcal{R}}(x, \mathcal{E}^{\mathcal{A}}(pp, x)) = 1] \geq \text{negl}(\lambda)$.
- **Succinctness** : An argument is succinct if the communication between $\mathcal{P}$ and $\mathcal{V}$ is sublinear in $|w|$.
- **Public coin** : An argument is public coin if each $\mathcal{V}$'s message $\mathcal{M} \leftarrow_{\$} \mathcal{C}$, for $\mathcal{C}$ is some fixed set.

Definition 7. An interactive argument $(\text{Gen}, \mathcal{P}, \mathcal{V})$ for $\mathcal{R}$ is **honest-verifier statistical zero-knowledge (HVSZK)** if there exists a PPT algorithm $S(x, z)$ called the simulator, running in time polynomial in $|x|$, such that for every $x \in \mathcal{R}$, $w \in \mathcal{R}_x$, and $z \in \{0, 1\}^*$, the statistical distance between the distributions $\text{tr}\langle \mathcal{P}(w), \mathcal{V}(z) \rangle(x)$ and $S(x, z)$ is $\text{negl}(\lambda)$.

Definition 8 (Witness-extended emulation [19,27,22]). A public coin interactive argument $(\text{Gen}, \mathcal{P}, \mathcal{V})$ for $\mathcal{R}$ has witness-extended emulation if for all deterministic polynomial time programs $\mathcal{P}^*$ there exists an expected polynomial time emulator $\mathcal{E}$ such that for all non-uniform polynomial time adversaries $\mathcal{A}$ and all $z_V \in \{0, 1\}^*$, the following probabilities differ by at most $\text{negl}(\lambda)$:

$$\Pr \left[\mathcal{A}(t, x) = 1 \mid \begin{array}{l} pp \leftarrow \text{Gen}(1^\lambda) \\ \wedge (x, z_P) \leftarrow \mathcal{A}(pp) \\ \wedge t \leftarrow \text{tr}\langle \mathcal{P}^*(z_P), \mathcal{V}(z_V) \rangle(x) \end{array} \right],$$

and

$$\Pr \left[\begin{array}{l} \mathcal{A}(t, x) = 1 \\ \wedge (\text{Accept}(t) = 1 \Rightarrow \text{Sat}_{\mathcal{R}}(x, w) = 1) \end{array} \mid \begin{array}{l} pp \leftarrow \text{Gen}(1^\lambda) \\ \wedge (x, z_P) \leftarrow \mathcal{A}(pp) \\ \wedge (t, w) \leftarrow \mathcal{E}^{\mathcal{P}^*(z_P)}(x) \end{array} \right].$$

The witness-extended emulation implies the soundness and knowledge soundness.

Definition 9 (Tree extractability (arguments) [22]). A $(2\mu+1)$ -move interactive protocol $(\mathcal{P}, \mathcal{V})$ with Verifier message space $\mathcal{C}$ is (W, ϵ) -**tree extractable** if there exists a PPT algorithm extracting a witness from $(w_1, \dots, w_\mu)$ -tree of accepting transcripts with failure probability $\leq \epsilon$, $\prod w_i \leq W$ and $\max_i(w_i) \leq \epsilon|\mathcal{C}|$.

Definition 10 (Tree extractability (reductions) [22]). We say an interactive protocol reducing $x \in \mathcal{R}$ to $x' \in \mathcal{R}'$ is (W, ϵ) -**tree extractable** if the composition of this arguments with a final $\mathcal{P}$ message revealing a witness w' for $x' \in \mathcal{R}'$ is a (W, ϵ) -tree extractable arguments for $\mathcal{R}$.

3 Hadamard Product Arguments

In this section, we introduce an efficient Hadamard product argument Π_{HP} characterized by transparency and sublinear verification time. Our Π_{HP} determines whether the Hadamard product $\mathbf{a} \circ \mathbf{b} = \mathbf{c} \circ \mathbf{d} \in \mathbb{G}_T^n$ holds solely based on commitments of vectors. Leveraging doubly homomorphic commitments via the AFGHO commitment [1], our scheme guarantees security under the SXDH assumption.

For clarity of presentation, the following description focuses on the core argument for the Hadamard product relation, omitting the specific details related to achieving zero-knowledge and hiding properties, and these extensions can be found in Appendix D. We first define the relation for Hadamard product arguments as follows:

Definition 11 (Relation for Hadamard Product Arguments). *Given commitment keys $\mathbf{ck}_1 \leftarrow \mathbb{G}_1^n$ and $\mathbf{ck}_2 \leftarrow \mathbb{G}_2^n$, the relation $\mathcal{R}_{n, \mathbf{ck}_1}^{HP}$ is defined as*

$$\left\{ (D_1, D_2, D'_1, D'_2 \in \mathbb{G}_T ; \mathbf{a}, \mathbf{c} \in \mathbb{G}_1^n, \mathbf{b}, \mathbf{d} \in \mathbb{G}_2^n) \left| \begin{array}{l} D_1 = \langle \mathbf{a}, \mathbf{ck}_2 \rangle, \\ D_2 = \langle \mathbf{ck}_1, \mathbf{b} \rangle, \\ D'_1 = \langle \mathbf{c}, \mathbf{ck}_2 \rangle, \\ D'_2 = \langle \mathbf{ck}_1, \mathbf{d} \rangle, \\ \mathbf{a} \circ \mathbf{b} = \mathbf{c} \circ \mathbf{d} \end{array} \right. \right\}$$

By the definition of the AFGHO commitment scheme, the inner product D_1 represents the commitment of vector $\mathbf{a}$ using the commitment key $\mathbf{ck}_2$, and similarly, D_2 , D'_1 , and D'_2 follow the same commitment structure.

To verify the Hadamard product relation $\mathbf{a} \circ \mathbf{b} = \mathbf{c} \circ \mathbf{d}$, we employ a polynomial-based approach. This method utilizes two polynomials, defined using a randomly chosen $r \leftarrow \mathbb{F}$:

$$X = \sum_{i=1}^n r^{i-1} e(a_i, b_i), \quad X' = \sum_{i=1}^n r^{i-1} e(c_i, d_i).$$

If these two polynomials are equal, then by the Schwartz–Zippel lemma, their coefficients must be equal with high probability, meaning that $e(a_i, b_i) = e(c_i, d_i)$ for all i .

To achieve this verification, the prover receives a challenge r from the verifier and computes new vectors $\mathbf{a}_r$ and $\mathbf{c}_r$ by applying the Hadamard product to $\mathbf{a}$ and $\mathbf{c}$, respectively. The verifier then checks whether the inner products $X = \langle \mathbf{a}_r, \mathbf{b} \rangle$ and $X' = \langle \mathbf{c}_r, \mathbf{d} \rangle$ are equal. Additionally, the verifier must ensure that the prover has correctly computed $\mathbf{a}_r$ and $\mathbf{c}_r$.

Our Hadamard Product Argument Π_{HP} consists of three subprotocols:

- **Reformatting** (see Section 3.1): The prover computes and commits to the vectors $\mathbf{a}_r$ and $\mathbf{c}_r$.
- **Elimination Folding** (see Section 3.2): The correctness of $\mathbf{a}_r$ and $\mathbf{c}_r$ is verified.
- **Pairing Check** (see Section 3.3): The verification is completed using pairing operations.

3.1 Reformatting Algorithm

In the reformatting Σ_{ref} , the prover computes new witness vectors $\mathbf{a}_r$ and $\mathbf{c}_r$ by incorporating a random challenge r . Additionally, to facilitate the subsequent folding step, the prover introduces new key vectors $\mathbf{k}$ and $\mathbf{k}'$ into the witness set.⁴ The vector $\mathbf{k}$ (and $\mathbf{k}'$) is initialized as the identity vector in $\mathbb{G}_1^n$.

⁴ Since each element of $\mathbf{a}_r$ is already multiplied by the random challenge r , it is not possible to incorporate and fold the commitment key $\mathbf{ck}_1$, unlike in Dory [22]. This makes verification more challenging for the verifier. To address this, we introduce new vectors $\mathbf{k}$ and $\mathbf{k}'$ to efficiently fold the key $\mathbf{ck}_1$.

Table 2: Symbols and Notations used in Sections 3 and 4

Symbol/Notation	Definition
$\mathbb{F}$	Finite field of prime order p
$(\mathbb{F}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g, h)$	Type III Pairing setup
n	Length of witness vectors ($n = 2^m$)
$\mathbf{a}, \mathbf{c} \in \mathbb{G}_1^n, \mathbf{b}, \mathbf{d} \in \mathbb{G}_2^n$	Witness vectors
$\mathbf{r} = (1, r, r^2, \dots, r^{n-1}) \in \mathbb{F}^n$	Random challenge vector ($r \leftarrow \mathbb{F}$)
$\mathbf{ck}_1 \in \mathbb{G}_1^n$	Commitment key for $\mathbf{b}, \mathbf{d}$
$\mathbf{ck}_2 \in \mathbb{G}_2^n$	Commitment key for $\mathbf{a}, \mathbf{c}$
$\mathbf{ck}_i = (\mathbf{ck}_1, \mathbf{ck}_2)$	Combined commitment keys
$\mathbf{k} \in \mathbb{G}_1^n$	Folding key
$\mathbf{a}_r \in \mathbb{G}_1^n$	Hadamard product: $\mathbf{a} \circ \mathbf{r}$
$\mathbf{a}_L, \mathbf{a}_R \in \mathbb{G}_1^{n/2}$	Left and right halves of $\mathbf{a}$
$\langle \mathbf{a}, \mathbf{b} \rangle \in \mathbb{G}_T$	Inner pairing product: $\prod_{i=1}^n e(a_i, b_i)$
$D_1 \in \mathbb{G}_T$	AFGHO commitment to $\mathbf{a}$ using $\mathbf{ck}_2$
$D_2 \in \mathbb{G}_T$	AFGHO commitment to $\mathbf{b}$ using $\mathbf{ck}_1$
$D'_1 \in \mathbb{G}_T$	AFGHO commitment to $\mathbf{c}$ using $\mathbf{ck}_2$
$D'_2 \in \mathbb{G}_T$	AFGHO commitment to $\mathbf{d}$ using $\mathbf{ck}_1$

Once these new vectors are set, the prover computes inner products and commitments and transmits them to the verifier. The verifier then derives the new relations $\mathcal{R}'_1$ and $\mathcal{R}'_2$.

The prover and verifier proceed with reformatting, following the steps outlined in Algorithm 2.

Initial Transmission [Line 1]. The prover transmits the inner pairing products C and C' , along with the commitments D_1, D_2, D'_1 , and D'_2 , which correspond to the commitments of the witness vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$, and $\mathbf{d}$, respectively.

Hadamard Products Computation [Lines 2-3]. The verifier selects a random challenge r and sends it to the prover. Using this challenge, the prover constructs the vector $\mathbf{r} = (1, r, r^2, \dots, r^{n-1})$ and computes the Hadamard products $\mathbf{a}_r = \mathbf{r} \circ \mathbf{a}$ and $\mathbf{c}_r = \mathbf{r} \circ \mathbf{c}$. Additionally, the prover initializes $\mathbf{k}$ and $\mathbf{k}'$ as identity vectors in $\mathbb{G}_1^n$.

Commitment Computation [Line 4]. The prover transmits the inner pairing products: $\langle \mathbf{a}_r, \mathbf{b} \rangle$, $\langle \mathbf{k}, \mathbf{b} \rangle$, $\langle \mathbf{c}_r, \mathbf{d} \rangle$, and $\langle \mathbf{k}', \mathbf{d} \rangle$. Additionally, the prover sends the AFGHO commitments of $\mathbf{a}_r$, $\mathbf{k}$, $\mathbf{c}_r$, and $\mathbf{k}'$ to the verifier.

3.2 Elimination Folding Algorithm

The verifier must check whether the equations $\mathbf{a}_r = \mathbf{r} \circ \mathbf{a}$ and $\mathbf{c}_r = \mathbf{r} \circ \mathbf{c}$ hold. Since these equations are symmetric, the same verification technique can be applied independently to each. Therefore, in this section, we introduce an algorithm for

Algorithm 2 Σ_{ref} : Reformatting Algorithm

Input : $\mathcal{R} = (D_1, D_2, D'_1, D'_2; \mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d})$
1: $\mathcal{P} \rightarrow \mathcal{V} : C = \langle \mathbf{a}, \mathbf{b} \rangle, \quad C' = \langle \mathbf{c}, \mathbf{d} \rangle$
2: $\mathcal{V} \rightarrow \mathcal{P} : r \leftarrow \mathbb{F}$
3: $\mathcal{P} : \mathbf{r} \leftarrow (1, r, r^2, \dots, r^{n-1}), \quad \mathbf{a}_r \leftarrow \mathbf{r} \circ \mathbf{a}, \quad \mathbf{c}_r \leftarrow \mathbf{r} \circ \mathbf{c},$
 $\quad \mathbf{k} \leftarrow \mathbf{1}, \quad \mathbf{k}' \leftarrow \mathbf{1} \in \mathbb{G}_1^n$
4: $\mathcal{P} \rightarrow \mathcal{V} : X = \langle \mathbf{a}_r, \mathbf{b} \rangle, \quad K = \langle \mathbf{k}, \mathbf{b} \rangle, \quad D_3 = \langle \mathbf{a}_r, \mathbf{ck}_2 \rangle, \quad D_4 = \langle \mathbf{k}, \mathbf{ck}_2 \rangle,$
 $\quad X' = \langle \mathbf{c}_r, \mathbf{b} \rangle, \quad K' = \langle \mathbf{k}', \mathbf{b} \rangle, \quad D'_3 = \langle \mathbf{c}_r, \mathbf{ck}_2 \rangle, \quad D'_4 = \langle \mathbf{k}', \mathbf{ck}_2 \rangle$
Output : $\mathcal{R}_1 = (C, X, K, D_{t \in [4]}, \mathbf{a}, \mathbf{b}, \mathbf{a}_r, \mathbf{k})$, and
 $\mathcal{R}_2 = (C', X', K', D'_{t \in [4]}; \mathbf{c}, \mathbf{d}, \mathbf{c}_r, \mathbf{k}')$

verifying $\mathbf{a}_r = \mathbf{r} \circ \mathbf{a}$ within the relation

$$\mathcal{R}_1 = (C, X, K, D_{t \in [4]}; \mathbf{a}, \mathbf{b}, \mathbf{a}_r, \mathbf{k}).$$

This algorithm reduces the vector length by half by folding $\mathbf{a}, \mathbf{b}, \mathbf{a}_r$, and $\mathbf{k}$. Unlike conventional folding techniques, our method eliminates the factor r that is multiplied with $\mathbf{a}_r$ during the folding process—an approach we refer to as *elimination folding*. As a result, we can confirm that $\mathbf{a}_r$ has correctly undergone the Hadamard product with $\mathbf{a}$ using $\mathbf{r}$.

We denote this elimination folding algorithm as Σ_{elim} , and the relation is defined as follows.

Definition 12 (Relation for Elimination Folding). *For a given commitment keys $\mathbf{ck}_1 \leftarrow \mathbb{G}_1^n$, and $\mathbf{ck}_2 \leftarrow \mathbb{G}_2^n$, and a random challenge $r \in \mathbb{F}$, the relation $\mathcal{R}_{r,n,\mathbf{ck}_1}^{\text{Elim}}$ is defined as follows:*

$$\left\{ (C, X, K, D_{t \in [4]} \in \mathbb{G}_T ; \mathbf{a}, \mathbf{a}_r, \mathbf{k} \in \mathbb{G}_1^n, \mathbf{b} \in \mathbb{G}_2^n) \left| \begin{array}{l} \mathbf{a}_r = (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{a}, \\ C = \langle \mathbf{a}, \mathbf{b} \rangle, \\ X = \langle \mathbf{a}_r, \mathbf{b} \rangle, \\ K = \langle \mathbf{k}, \mathbf{b} \rangle, \\ D_1 = \langle \mathbf{a}, \mathbf{ck}_2 \rangle, \\ D_2 = \langle \mathbf{ck}_1, \mathbf{b} \rangle, \\ D_3 = \langle \mathbf{a}_r, \mathbf{ck}_2 \rangle, \\ D_4 = \langle \mathbf{k}, \mathbf{ck}_2 \rangle \end{array} \right. \right\}$$

After reformatting, the prover and verifier obtain a new relation:

$$\mathcal{R}_1 = (C, X, K, D_{t \in [4]}; \mathbf{a}, \mathbf{b}, \mathbf{a}_r, \mathbf{k}) \in \mathcal{R}_{r,n,\mathbf{ck}_1}^{\text{Elim}}.$$

In Σ_{elim} , the prover performs key folding by combining the commitment keys $\mathbf{ck}_1$ and $\mathbf{ck}_2$ with the key vector $\mathbf{k}$ and the message vector $\mathbf{b}$, respectively. The folding process involves two verifier challenges, α and β , which are used to combine the left and right halves of vectors. Specifically, α is used to merge the left and right halves, while β separates the witness vectors from the commitment keys. By properly incorporating α and β into the folding process, the commitment key terms remain distinguishable from the witness vectors. Since this

procedure is embedded in the folding step, the verifier can delegate the entire commitment key computation to the prover, ensuring that the prover correctly follows the key folding procedure.

However, since the vector $\mathbf{a}$ has already been multiplied by r during reformatting, performing key folding at this stage would prevent the elimination of r in the elimination folding step. This is because the left and right halves of $\mathbf{a}$ are distinguished not only by their degrees in α and r but also by the inclusion of β . To address this, we separate the key folding term β from $\mathbf{a}$ and $\mathbf{a}_r$ and instead assign this role to the key vector $\mathbf{k}$.

The prover and verifier proceed with elimination folding, following the steps outlined in Algorithm 3.

Algorithm 3 Σ_{elim} : Elimination Folding Algorithm

Precompute : $\mathbf{ck}'_1 \leftarrow \mathbf{ck}_{1L}, \quad \mathbf{ck}'_2 \leftarrow \mathbf{ck}_{2L}, \quad \chi = \langle \mathbf{ck}_1, \mathbf{ck}_2 \rangle,$
for $I \in \{L, R\}, \quad \Delta_{1I} = \langle \mathbf{ck}_{1I}, \mathbf{ck}'_2 \rangle, \Delta_{2I} = \langle \mathbf{ck}'_1, \mathbf{ck}_{2I} \rangle$

Input : $\mathcal{R}_1 = (C, X, K, D_{t \in [4]}; \mathbf{a}, \mathbf{b}, \mathbf{a}_r, \mathbf{k}) \in \mathcal{R}_{r,n,\mathbf{ck}_i}^{\text{Elim}}$

1: $\mathcal{P} \rightarrow \mathcal{V}$: for $I \in \{L, R\},$
 $X_I = \langle \mathbf{a}_{rI}, \mathbf{b}_I \rangle, \quad D_{3I} = \langle \mathbf{a}_{rI}, \mathbf{ck}_{2I} \rangle,$
 $D'_{1I} = \langle \mathbf{a}_I, \mathbf{ck}'_2 \rangle, \quad D'_{2I} = \langle \mathbf{ck}'_1, \mathbf{b}_I \rangle, \quad D'_{3I} = \langle \mathbf{a}_{rI}, \mathbf{ck}'_2 \rangle, \quad D'_{4I} = \langle \mathbf{k}_I, \mathbf{ck}'_2 \rangle$

2: $\mathcal{V} \rightarrow \mathcal{P}$: $\beta \leftarrow \mathbb{F}$

3: $\mathcal{P} : \mathbf{b}^* \leftarrow \mathbf{b} + \beta^{-1} \mathbf{ck}_2, \quad \mathbf{k}^* \leftarrow \mathbf{k} + \beta \mathbf{ck}_1$

4: $\mathcal{P} \rightarrow \mathcal{V}$: $C_+ = \langle \mathbf{a}_L, \mathbf{b}_R^* \rangle, \quad C_- = \langle \mathbf{a}_R, \mathbf{b}_L^* \rangle,$
 $X_+ = \langle \mathbf{a}_{rL}, \mathbf{b}_R^* \rangle, \quad X_- = \langle \mathbf{a}_{rR}, \mathbf{b}_L^* \rangle,$
 $K_+ = \langle \mathbf{k}_L^*, \mathbf{b}_R^* \rangle, \quad K_- = \langle \mathbf{k}_R^*, \mathbf{b}_L^* \rangle$

5: $\mathcal{V} \rightarrow \mathcal{P}$: $\alpha \leftarrow \mathbb{F}$

6: $\mathcal{P} : \mathbf{a}' \leftarrow \alpha \mathbf{a}_L + \mathbf{a}_R, \quad \mathbf{b}' \leftarrow \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^*,$
 $\mathbf{a}'_r \leftarrow \alpha \mathbf{a}_{rL} + r^{-\frac{n}{2}} \mathbf{a}_{rR}, \quad \mathbf{k}' \leftarrow \alpha \mathbf{k}_L^* + \mathbf{k}_R^*$

7: $\mathcal{V}$: Check if $X = X_L + X_R, \quad D_3 = D_{3L} + D_{3R}$
 $C' \leftarrow C + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_-$
 $X' \leftarrow X_L + r^{-\frac{n}{2}} X_R + \beta^{-1} (D_{3L} + r^{-\frac{n}{2}} D_{3R}) + \alpha X_+ + \alpha^{-1} r^{-\frac{n}{2}} X_-$
 $K' \leftarrow K + \chi + \beta D_2 + \beta^{-1} D_4 + \alpha K_+ + \alpha^{-1} K_-$
 $D'_1 \leftarrow \alpha D'_{1L} + D'_{1R},$
 $D'_2 \leftarrow \alpha^{-1} D'_{2L} + D'_{2R} + \alpha^{-1} \beta^{-1} \Delta_{2L} + \beta^{-1} \Delta_{2R}$
 $D'_3 \leftarrow \alpha D'_{3L} + r^{-\frac{n}{2}} D'_{3R},$
 $D'_4 \leftarrow \alpha D'_{4L} + D'_{4R} + \alpha \beta \Delta_{1L} + \beta \Delta_{1R}$

Output : $\mathcal{R}'_1 = (C', X', K', D'_{t \in [4]}; \mathbf{a}', \mathbf{b}', \mathbf{a}'_r, \mathbf{k}') \in \mathcal{R}_{r,n/2,\mathbf{ck}'_i}^{\text{Elim}}$

Initialization [Line 1]. The prover computes the inner pairing products of the left and right halves of the vectors $\mathbf{a}, \mathbf{b}, \mathbf{a}_r$, and $\mathbf{k}$ with the corresponding commitment keys. These intermediate values are sent to the verifier for subsequent verification.

Witness-Key Combination [Lines 2-3]. The verifier provides a random challenge β . The prover then uses β to combine $\mathbf{b}$ with $\mathbf{ck}_2$, forming a new vector

$\mathbf{b}^*$, and $\mathbf{k}$ with $\mathbf{ck}_1$, forming $\mathbf{k}^*$. This step prepares the vectors for folding and shifts the responsibility of commitment key computations to the prover.

Computation of Intermediate Values [Line 4]. The prover computes the intermediate values $C_{\pm}, X_{\pm}, K_{\pm}$, which represent inner pairing products involving the left and right halves of $\mathbf{a}, \mathbf{a}_r, \mathbf{b}^*$, and $\mathbf{k}^*$. These values are transmitted to the verifier and used in the final verification step.

Vector Folding [Lines 5-6]. The verifier provides another random challenge α . The prover then uses α and r to fold the vectors $\mathbf{a}, \mathbf{b}^*, \mathbf{a}_r$, and $\mathbf{k}^*$, reducing their length by half. The key operation here is the elimination folding of $\mathbf{a}_r$, where multiplication by $r^{-n/2}$ ensures the degree of r is properly adjusted.

Verification [Line 7]. The verifier checks the consistency of the values received from the prover and computes updated values $C', X', K', D'_{t \in [4]}$, which will be used as inputs for the next iteration (or as final verification values if this is the last round). These computations involve combining the received intermediate values with the challenges α and β , as defined in the algorithm.

Theorem 1. *For a commitment keys $\mathbf{ck}'_1 \leftarrow \mathbb{G}_1^{n/2}$, and $\mathbf{ck}'_2 \leftarrow \mathbb{G}_2^{n/2}$, Σ_{elim} is a HVSZK, public-coin, succinct interactive argument of knowledge for $\mathcal{R}_{r,n/2,\mathbf{ck}_1}^{Elim}$ with $(9, 12/|\mathbb{F}|)$ -tree extractability under SXDH.*

Proof. First and foremost, succinctness and the public coin property are immediate. Refer to Theorem 6 in the Appendix for the validity of HVSZK.

Completeness : Using the properties of AFGHO commitment, completeness can be easily verified as follows:

$$\begin{aligned} C' &= \langle \mathbf{a}', \mathbf{b}' \rangle = \langle \alpha \mathbf{a}_L + \mathbf{a}_R, \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^* \rangle = C + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_-, \\ X' &= \langle \mathbf{a}_r', \mathbf{b}' \rangle = \langle \alpha \mathbf{a}_{rL} + r^{-\frac{n}{2}} \mathbf{a}_{rR}, \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^* \rangle \\ &= X_L + r^{-\frac{n}{2}} X_R + \beta^{-1} (D_{3L} + r^{-\frac{n}{2}} D_{3R}) + \alpha X_+ + \alpha^{-1} r^{-\frac{n}{2}} X_-, \\ K' &= \langle \mathbf{k}', \mathbf{b}' \rangle = \langle \alpha \mathbf{k}_L^* + \mathbf{k}_R^*, \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^* \rangle = K + \chi + \beta D_2 + \beta^{-1} D_4 + \alpha K_+ + \alpha^{-1} K_-, \end{aligned}$$

$$\begin{aligned} D'_1 &= \langle \mathbf{a}', \mathbf{ck}'_2 \rangle = \langle \alpha \mathbf{a}_L + \mathbf{a}_R, \mathbf{ck}'_2 \rangle = \alpha D'_{1L} + D'_{1R}, \\ D'_2 &= \langle \mathbf{ck}'_1, \mathbf{b}' \rangle = \langle \mathbf{ck}'_1, \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^* \rangle = \alpha^{-1} D'_{2L} + D'_{2R} + \alpha^{-1} \beta^{-1} \Delta_{2L} + \beta^{-1} \Delta_{2R}, \\ D'_3 &= \langle \mathbf{a}_r', \mathbf{ck}'_2 \rangle = \langle \alpha \mathbf{a}_{rL} + r^{-\frac{n}{2}} \mathbf{a}_{rR}, \mathbf{ck}'_2 \rangle = \alpha D'_{3L} + r^{-\frac{n}{2}} D'_{3R}, \\ D'_4 &= \langle \mathbf{k}', \mathbf{ck}'_2 \rangle = \langle \alpha \mathbf{k}_L^* + \mathbf{k}_R^*, \mathbf{ck}'_2 \rangle = \alpha D'_{4L} + D'_{4R} + \alpha \beta \Delta_{1L} + \beta \Delta_{1R}. \end{aligned}$$

Tree extractability : From definition 9, we start with $\mu = 2$ and set $w_1 = w_2 = 3$. Thus, we have a tree of accepting transcripts for 3 values of β , and each β has 3 corresponding values of α . The failure scenario arises if any of these challenges are 0, which happens with a probability of $\leq 9/|\mathbb{F}|$. Next, we interpolate $C_{\pm}, X_{\pm}$, and $K_{\pm}$ as Laurent polynomials in $\mathbb{G}_T[\beta, \beta^{-1}]$ of degree 1 and order 1. Moreover, we perform interpolation for $\mathbf{a}'$ and $\mathbf{a}_r'$ as polynomials in $\mathbb{G}_T[\alpha]$ of degree 1, and $\mathbf{b}'$ and $\mathbf{k}'$ as Laurent polynomials in $\mathbb{G}_T[\alpha, \beta]$ of degree

1 and order 1. Let $s = r^{-\frac{n}{2}}$. For each leaf, considering $D'_3 = \alpha D'_{3L} + s D'_{3R} = \langle \mathbf{a}_{\mathbf{r}'}(\alpha), \mathbf{c}\mathbf{k}'_2 \rangle$, we deduce that $\mathbf{a}_{\mathbf{r}'}(\alpha) = \alpha \mathbf{a}_{\mathbf{r}L} + s \mathbf{a}_{\mathbf{r}R}$ by comparing the coefficients corresponding to α . Additionally, deriving from the equation $D'_4 = \alpha D'_{4L} + D'_{4R} + \alpha \beta \Delta_{1L} + \beta \Delta_{1R} = \langle \mathbf{k}', \mathbf{c}\mathbf{k}'_2 \rangle$, we can deduce that $\mathbf{k}'(\alpha, \beta) = \alpha \mathbf{k}_L + \mathbf{k}_R + \alpha \beta \mathbf{c}\mathbf{k}_{1L} + \beta \mathbf{c}\mathbf{k}_{1R}$. It's worth highlighting that from D'_1 and D'_2 , the following relations emerge: $\mathbf{a}'(\alpha) = \alpha \mathbf{a}_L + \mathbf{a}_R$ and $\mathbf{b}'(\alpha, \beta) = \alpha^{-1} \mathbf{b}_L + \mathbf{b}_R + \alpha^{-1} \beta^{-1} \mathbf{c}\mathbf{k}_{2L} + \beta^{-1} \mathbf{c}\mathbf{k}_{2R}$. Finally, we proceed to substitute these expressions into C', X' , and K' :

$$\begin{aligned} C' &= C + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_- = \langle \mathbf{a}'(\alpha), \mathbf{b}'(\alpha, \beta) \rangle \\ &= \langle \mathbf{a}, \mathbf{b}^* \rangle + \alpha \langle \mathbf{a}_L, \mathbf{b}_R^* \rangle + \alpha^{-1} \langle \mathbf{a}_R, \mathbf{b}_L^* \rangle \\ &= \langle \mathbf{a}, \mathbf{b} \rangle + \beta^{-1} \langle \mathbf{a}, \mathbf{c}\mathbf{k}_2 \rangle + \alpha \langle \mathbf{a}_L, \mathbf{b}_R^* \rangle + \alpha^{-1} \langle \mathbf{a}_R, \mathbf{b}_L^* \rangle, \end{aligned}$$

$$\begin{aligned} X' &= X_L + s X_R + \beta^{-1} (D_{3L} + s D_{3R}) + \alpha X_+ + \alpha^{-1} s X_- = \langle \mathbf{a}_{\mathbf{r}'}(\alpha), \mathbf{b}'(\alpha, \beta) \rangle \\ &= \langle \alpha \mathbf{a}_{\mathbf{r}L} + s \mathbf{a}_{\mathbf{r}R}, \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^* \rangle \\ &= \langle \mathbf{a}_{\mathbf{r}L}, \mathbf{b}_L \rangle + s \langle \mathbf{a}_{\mathbf{r}R}, \mathbf{b}_R \rangle + \beta^{-1} (\langle \mathbf{a}_{\mathbf{r}L}, \mathbf{c}\mathbf{k}_{2L} \rangle + s \langle \mathbf{a}_{\mathbf{r}R}, \mathbf{c}\mathbf{k}_{2R} \rangle) + \alpha \langle \mathbf{a}_{\mathbf{r}L}, \mathbf{b}_R^* \rangle + \alpha^{-1} s \langle \mathbf{a}_{\mathbf{r}R}, \mathbf{b}_L^* \rangle, \end{aligned}$$

$$\begin{aligned} K' &= K + \chi + \beta D_2 + \beta^{-1} D_4 + \alpha K_+ + \alpha^{-1} K_- = \langle \mathbf{k}'(\alpha, \beta), \mathbf{b}'(\alpha, \beta) \rangle \\ &= \langle \alpha \mathbf{k}_L^* + \mathbf{k}_R^*, \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^* \rangle \\ &= \langle \mathbf{k}, \mathbf{b} \rangle + \chi + \beta \langle \mathbf{c}\mathbf{k}_1, \mathbf{b} \rangle + \beta^{-1} \langle \mathbf{k}, \mathbf{c}\mathbf{k}_2 \rangle + \alpha \langle \mathbf{k}_L^*, \mathbf{b}_R^* \rangle + \alpha^{-1} \langle \mathbf{k}_R^*, \mathbf{b}_L^* \rangle. \end{aligned}$$

The first equation, $C + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_- = \langle \mathbf{a}, \mathbf{b} \rangle + \beta^{-1} \langle \mathbf{a}, \mathbf{c}\mathbf{k}_2 \rangle + \alpha \langle \mathbf{a}_L, \mathbf{b}_R^* \rangle + \alpha^{-1} \langle \mathbf{a}_R, \mathbf{b}_L^* \rangle$, demonstrates that each term is separated by the random challenges α and β . According to the SXDH assumption, Lemma 1 and Lemma 2 confirm that these terms are indeed equal. Therefore, by comparing the coefficients of 1, α , α^{-1} , β , and β^{-1} (where s is a known value and easily factored), we obtain the following expressions:

$$\begin{aligned} C &= \langle \mathbf{a}, \mathbf{b} \rangle, \\ X &= X_L + X_R = \langle \mathbf{a}_{\mathbf{r}L}, \mathbf{b}_L \rangle + \langle \mathbf{a}_{\mathbf{r}R}, \mathbf{b}_R \rangle = \langle \mathbf{a}_{\mathbf{r}}, \mathbf{b} \rangle, \\ K &= \langle \mathbf{k}, \mathbf{b} \rangle, \\ D_1 &= \langle \mathbf{a}, \mathbf{c}\mathbf{k}_2 \rangle, \\ D_2 &= \langle \mathbf{c}\mathbf{k}_1, \mathbf{b} \rangle, \\ D_3 &= D_{3L} + D_{3R} = \langle \mathbf{a}_{\mathbf{r}L}, \mathbf{c}\mathbf{k}_{2L} \rangle + \langle \mathbf{a}_{\mathbf{r}R}, \mathbf{c}\mathbf{k}_{2R} \rangle = \langle \mathbf{a}_{\mathbf{r}}, \mathbf{c}\mathbf{k}_2 \rangle, \\ D_4 &= \langle \mathbf{k}, \mathbf{c}\mathbf{k}_2 \rangle \end{aligned}$$

The final concept we're going to establish is $\mathbf{a}_{\mathbf{r}} = \mathbf{r} \circ \mathbf{a}$. We are already aware of the fact that $\mathbf{a}_{\mathbf{r}'} = \alpha \mathbf{a}_{\mathbf{r}L} + s \mathbf{a}_{\mathbf{r}R}$ and $\mathbf{a}' = \alpha \mathbf{a}_L + \mathbf{a}_R$. Based on the assumption, $\mathbf{a}_{\mathbf{r}'} = \mathbf{r}' \circ \mathbf{a}'$, where $\mathbf{r}' = \mathbf{r}_L = (1, r, \dots, r^{\frac{n}{2}-1})$. This yields:

$$\alpha \mathbf{a}_{\mathbf{r}L} + s \mathbf{a}_{\mathbf{r}R} = \mathbf{r}' \circ (\alpha \mathbf{a}_L + \mathbf{a}_R) = \mathbf{r}' \circ \alpha \mathbf{a}_L + \mathbf{r}' \circ \mathbf{a}_R.$$

By comparing coefficients involving α , we conclude that $\mathbf{a}_{rL} = \mathbf{r}' \circ \mathbf{a}_L$ and $s\mathbf{a}_{rR} = \mathbf{r}' \circ \mathbf{a}_R$. Additionally, $\mathbf{a}_{rR} = s^{-1}(\mathbf{r}' \circ \mathbf{a}_R) = (r^{\frac{n}{2}}, r^{\frac{n}{2}+1}, \dots, r^{n-1}) \circ \mathbf{a}_R$. Hence, we establish that $\mathbf{a}_r = \mathbf{r} \circ \mathbf{a}$. Therefore, the provided witness $(\mathbf{a}, \mathbf{b}, \mathbf{a}_r, \mathbf{k})$ satisfies the desired properties for $(C, X, K, D_{t \in [4]})$. $\square$

3.3 Pairing Check Algorithm

Since the elimination folding process reduces the length of witness vectors by half at each step, we perform Σ_{elim} a total of $m = \log n$ times. After completing this process, the prover and verifier obtain the reduced relation:

$$\mathcal{R}'_1 = (C, X, K, D_{t \in [4]}; a, b, a_r, k) \in \mathcal{R}_{r,1,ck_i}^{\text{Elim}}.$$

If the prover has correctly computed $\mathbf{a}_r$, all challenge terms r multiplied in $\mathbf{a}_r$ will be eliminated through elimination folding. Consequently, we must have $a = a_r \in \mathbb{G}_1$, which can be verified by checking the pairing equations:

$$e(a, b) = e(a_r, b), \quad e(a, ck_2) = e(a_r, ck_2).$$

To efficiently verify these equations, the prover combines a and a_r using a challenge, and the verifier then applies this combination to the pairing operation. As a result, the verifier can confirm both equations using a single pairing operation, demonstrating that our Hadamard product proof is designed for efficiency, requiring only two pairing operations in total.

We denote this verification algorithm as Σ_{check} . The prover and verifier proceed with the pairing check, following the steps outlined in Algorithm 4.

Witness Randomization [Lines 1-2]. The verifier generates a random challenge $c \leftarrow \$ \mathbb{F}$ and sends it to the prover. Using c , the prover randomizes the witnesses a, b, a_r, k and computes the values $E_1 \in \mathbb{G}_1$ and $E_2 \in \mathbb{G}_2$, which are then sent to the verifier. In particular, E_1 is defined as $E_1 = ca + c^3a + c^5a_r + c^7k$. By separating the identical a terms using the challenges c and c^3 , the verifier expands the pairing equation and substitutes C and X into the resulting expressions for C to check if $C = X$ (similarly for $D_1 = D_3$).

Final Verification [Line 3]. To simultaneously verify both the correctness of witness computation and key folding, the verifier introduces an additional random challenge $d \leftarrow \$ \mathbb{F}$ and modifies the pairing check as follows:

$$e(E_1 + d \cdot ck_1, E_2 + d^{-1} \cdot ck_2).$$

When expanding the pairing equation, the term $c^4e(a, b)$ should equal c^4C if the reduction algorithm has been correctly applied. However, it must also hold that $c^4e(a_r, b) = c^4X$. Thus, the verifier substitutes C with X and D_1 with D_3 in the final verification equation.

If the pairing equation holds, it confirms that $a_r = a$, ensuring the correct execution of the elimination folding process—i.e., verifying that the vector $\mathbf{a}_r$ correctly satisfies $\mathbf{a}_r = \mathbf{r} \circ \mathbf{a}$.

Algorithm 4 Σ_{check} : Pairing Check Algorithm**Precompute** : $\chi = e(ck_1, ck_2)$ **Input** : $\mathcal{R}'_1 = (C, X, K, D_{t \in [4]}; a, b, a_r, k) \in \mathcal{R}_{r,1,ck_i}^{\text{Elim}}$ 1: $\mathcal{V} \rightarrow \mathcal{P} : c \leftarrow \mathbb{F}$ 2: $\mathcal{P} \rightarrow \mathcal{V} : E_1 \leftarrow ca + c^3a + c^5a_r + c^7k, \quad E_2 \leftarrow cb$ 3: $\mathcal{V} : d \leftarrow \mathbb{F}$, accept if :

$$\begin{aligned}
& e(E_1 + d \cdot ck_1, E_2 + d^{-1} \cdot ck_2) \\
&= \chi + c^2C + c^4(1 + c^2)X + c^8K \\
&\quad + cd^{-1}D_1 + c^3d^{-1}(1 + c^2)D_3 + c^7d^{-1}D_4
\end{aligned}$$

Theorem 2. For a given commitment keys $ck_1 \leftarrow \mathbb{G}_1$, $ck_2 \leftarrow \mathbb{G}_2$, Σ_{check} is a HVSZK, public-coin, succinct interactive argument of knowledge for $\mathcal{R}_{r,1,ck_i}^{\text{Elim}}$ with $(27, 27/|\mathbb{F}|)$ -tree extractability under SXDH.

Proof. Succinctness and the public coin property are evident. Refer to theorem 7 in the Appendix for the validity of HVSZK.

Completeness : Given that $\mathbf{r} = (r^0)$ in $\mathcal{R}_{r,1,ck_i}^{\text{Elim}}$, we have $a = a_r$. This implies $X = e(a_r, b) = e(a, b) = C$ and $D_1 = e(a, ck_2) = e(a_r, ck_2) = D_3$ in $\mathcal{R}_{r,1,ck_i}^{\text{Elim}}$. Hence, completeness is demonstrated as follows:

$$\begin{aligned}
& e(E_1 + d \cdot ck_1, E_2 + d^{-1} \cdot ck_2) \\
&= e(ca + c^3a + c^5a_r + c^7k + d \cdot ck_1, cb + d^{-1} \cdot ck_2) \\
&= \chi + c^2e(a, b) + c^4e(a, b) + c^6e(a_r, b) + c^8e(k, b) \\
&\quad + cde(ck_1, b) + cd^{-1}e(a, ck_2) + c^3d^{-1}e(a, ck_2) \\
&\quad + c^5d^{-1}e(a_r, ck_2) + c^7d^{-1}e(k, ck_2) \\
&= \chi + c^2(C + c^2X + c^4X + c^6K) + cdD_2 + cd^{-1}(D_1 + c^2D_3 + c^4D_3 + c^6D_4)
\end{aligned}$$

Tree extractability : In our scenario, we set $\mu = 2$ and designate $w_1 = 9$ and $w_2 = 3$. Consequently, we find ourselves with a structure comprising accepting transcripts for 9 distinct values of c , and within each c value, there exists 3 corresponding accepting values of d . The event of failure arises if any of these 27 d values turns out to be 0. This unfortunate outcome transpires with a probability that is $\leq 27/|\mathbb{F}|$. Across all the presented transcripts, it remains constant that C, X, K , and $D_{t \in [4]}$ hold fixed. Furthermore, it is possible to interpolate E_1 and E_2 as polynomials of degree 8 concerning c . This interpolation is expressed as: $E_1(c) = d_1 + ca + c^3a + c^5a_r + c^7k + \sum_{i=1}^4 c^{2i}U_i$ and $E_2(c) = d_2 + cb + \sum_{i=1}^7 c^{j+1}U_j$. For each distinct c value, the conclusive verification encompasses terms reliant solely on d in the forms of $d, 1, d^{-1}$. This verification takes the shape of an equality check involving Laurent polynomials marked by degrees and orders of 1. The distinctive aspect here is that this discrepancy disappears for three distinct selections of d , thereby compelling the coefficients for each degree to be equal

independently. Thus, for each of the 9 challenge values of c :

$$e(E_1(c), ck_2) = P_1 + cD_1 + (c^3 + c^5)D_3 + c^7D_4 \quad (1)$$

$$e(ck_1, E_2(c)) = P_2 + cD_2 \quad (2)$$

$$e(E_1(c), E_2(c)) = \sum_{i=1}^3 c^{2i-1}Q_i + c^7P_3 + c^2C + (c^4 + c^6)X + c^8K \quad (3)$$

Equation (1) represents an equality of polynomials in $\mathbb{G}_T[c]$ of degree 8, which is valid at 9 points. Therefore, the coefficients on both the left-hand side and the right-hand side must be equal. Let's expand the coefficients to reveal their equivalence:

$$\begin{aligned} e(E_1(c), ck_2) &= e\left(d_1 + ca + c^3a + c^5a_r + c^7k + \sum c^{2i}U_i, ck_2\right) \\ &= e(d_1, ck_2) + ce(a, ck_2) + c^3e(a, ck_2) + c^5e(a_r, ck_2) \\ &\quad + c^7e(k, ck_2) + \sum c^{2i}e(U_i, ck_2) \\ &= P_1 + cD_1 + c^3D_3 + c^5D_3 + c^7D_4 + \sum c^{2i}e(U_i, ck_2) \end{aligned}$$

From Lemma 1 and Lemma 2, we deduce that $D_1 = e(a, ck_2)$, $D_3 = e(a_r, ck_2) = e(a, ck_2)$, $D_4 = e(k, ck_2)$, and $e(U_i, ck_2) = 0$ for all $i \in [4]$. Similarly, by comparing the coefficients of $e(ck_1, E_2(c))$ and $P_2 + cD_2$, we deduce that $D_2 = e(ck_1, b)$ and $e(ck_1, V_j) = 0$ for all $j \in [7]$, under the SXDH assumption.

As a result, we obtain the relationships: $a = a_r$, $U_i = V_j = r_{U_i} = r_{V_j} = 0$, and thus $E_1(c) = d_1 + ca + c^3a + c^5a_r + c^7k$ and $E_2(c) = d_2 + cb$. This allows us to identify tuple (a, b, a_r, k) that adhere to our constraints for (D_1, D_2, D_3, D_4) . Substituting the expressions for E_1 and E_2 into Equation (3), we get:

$$\begin{aligned} e(E_1(c), E_2(c)) &= e(d_1 + ca + c^3a + c^5a_r + c^7k, d_2 + cb) \\ &= e(d_1, d_2) + c(e(d_1, b) + e(a, d_2)) + c^3e(a, d_2) + c^5e(a_r, d_2) \\ &\quad + c^7e(k, d_2) + c^2e(a, b) + c^4e(a, b) + c^6e(a_r, b) + c^8e(k, d_2) \\ &= R + cQ_1 + c^3Q_2 + c^5Q_3 + c^7P_3 + c^2C + c^4X + c^6X + c^8K \end{aligned}$$

Since this equality holds in $\mathbb{G}_T[c]$ at 9 distinct values, we can equate the coefficients of the powers of c on both sides. So, we obtain the following equations: $C = e(a, b)$, $X = e(a, b) = e(a_r, b)$, $K = e(k, b)$. From these equations, we have (a, b) , (a_r, b) , and (k, b) satisfying our constraints on C, X, K . Additionally, since $a = a_r$, a_r fulfills the requirement $a_r = r^0 \circ a$. We conclude that (a, b, a_r, k) forms a valid witness for $(C, X, K, D_{t \in [4]}) \in \mathcal{R}_{r,1,ck_i}^{Elim}$. $\square$

3.4 Hadamard Product Arguments

Since the commitment keys $\mathbf{ck}_1$ and $\mathbf{ck}_2$ are publicly available, both the prover and verifier can precompute the inner products $\Delta_{1L}, \Delta_{1R}, \Delta_{2L}, \Delta_{2R}, \chi$ required for the elimination folding algorithm Σ_{elim} .

This precomputation allows the verifier to perform only $\mathcal{O}(1)$ group operations using commitments in each elimination folding step, thereby reducing the overall verification cost to logarithmic complexity.

Our Π_{HP} takes the initial relation $\mathcal{R}$ as input and executes Σ_{ref} to derive the elimination folding relations $(\mathcal{R}_1, \mathcal{R}_2)$. The verifier checks whether X and X' are equal. The prover and verifier then proceed with Σ_{elim} and Σ_{check} using these relations to complete the proof verification process.

The detailed protocol is presented in Algorithm 5.

Algorithm 5 Π_{HP} : Hadamard Product Arguments

Precompute : for all $0 \leq j \leq m-1$, compute

$$\mathbf{ck}_{1,j+1} = (\mathbf{ck}_{1,j})_L, \quad \mathbf{ck}_{2,j+1} = (\mathbf{ck}_{2,j})_L,$$

for all $0 \leq j \leq m$, compute

$$\chi_j = \langle \mathbf{ck}_{1,j}, \mathbf{ck}_{2,j} \rangle,$$

and for all $0 \leq j \leq m-1$, compute

$$\Delta_{1L,j} = \langle (\mathbf{ck}_{1,j})_L, \mathbf{ck}_{2,j+1} \rangle, \quad \Delta_{2L,j} = \langle \mathbf{ck}_{1,j+1}, (\mathbf{ck}_{2,j})_L \rangle,$$

$$\Delta_{1R,j} = \langle (\mathbf{ck}_{1,j})_R, \mathbf{ck}_{2,j+1} \rangle, \quad \Delta_{2R,j} = \langle \mathbf{ck}_{1,j+1}, (\mathbf{ck}_{2,j})_R \rangle$$

Input : $\mathcal{R} = (D_1, D_2, D'_1, D'_2; \mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}) \in \mathcal{R}_{2^m, \mathbf{ck}}^{\text{HP}}$

1: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{ref}}(\mathcal{R}) \rightarrow (\mathcal{R}_1, \mathcal{R}_2) \in (\mathcal{R}_{r, 2^m, \mathbf{ck}_{1,0}}^{\text{Elim}})^2$

2: $\mathcal{V} : \text{Check if } X = X'$

For $1 \leq j \leq m$:

3: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{elim}}(\mathcal{R}_1) \rightarrow \mathcal{R}'_1 \in \mathcal{R}_{r, 2^{m-j}, \mathbf{ck}_{1,j}}^{\text{Elim}}$

$$\Sigma_{\text{elim}}(\mathcal{R}_2) \rightarrow \mathcal{R}'_2 \in \mathcal{R}_{r, 2^{m-j}, \mathbf{ck}_{1,j}}^{\text{Elim}}$$

4: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{check}}(\mathcal{R}'_1) \rightarrow \text{result}_1$

$$\Sigma_{\text{check}}(\mathcal{R}'_2) \rightarrow \text{result}_2$$

5: $\mathcal{V} : \text{Accept if } (\text{result}_1 = 1 \wedge \text{result}_2 = 1)$

Theorem 3. Let $n = 2^m$. For a commitment keys $\mathbf{ck}_{1,0} \leftarrow \$ \mathbb{G}_1^n$, and $\mathbf{ck}_{2,0} \leftarrow \$ \mathbb{G}_2^n$, the Π_{HP} is a HVSZK, public-coin, succinct interactive argument of knowledge for $\mathcal{R}_{n, \mathbf{ck}_{1,0}}^{\text{HP}}$ with $(6 \cdot 9^{m+1}, 57 \cdot 9^m / |\mathbb{F}|)$ -tree extractability under SXDH.

Proof. This follows directly from **Theorem 1** and **Theorem 2**. In this protocol, we have $(W, \epsilon) = (9, 12)$ for $m = \log n$ rounds of Σ_{elim} and $(27, 27)$ for 1 round of Σ_{check} . Therefore, W equals $27 \cdot 9^m$, while the error ϵ amounts to $12(1 + 9 + \dots + 9^{m-1}) + 27 \cdot 9^m \approx 28.5 \cdot 9^m$, as outlined in Lemma 4 of [22]. We use Σ_{elim} and Σ_{check} on the left and right sides, respectively, so ultimately, $(W, \epsilon) = 2 \cdot (27 \cdot 9^m, 28.5 \cdot 9^m) = (6 \cdot 9^{m+1}, 57 \cdot 9^m)$. $\square$

4 Complexity Analysis

For vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}$ of length n , our Hadamard Product Argument Π_{HP} exhibits the following asymptotic complexity. We denote F as the cost of a field scalar operation, G_1, G_2, G_T as the costs of group multiplications in $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$, respectively, and P as the cost of a pairing operation. The cost of precomputations is omitted from our analysis.

Prover Complexity. The prover's workload is divided into three main phases: Σ_{ref} , Σ_{elim} , and Σ_{check} .

During the Σ_{ref} phase, the prover computes the inner pairing products $C, C', X, K, D_3, D_4, X', K', D'_3, D'_4$, requiring $\mathcal{O}(n)P$. The computation of the random vector $\mathbf{r}$ involves $\mathcal{O}(n)F$, while constructing the Hadamard product vectors $\mathbf{a}_{\mathbf{r}}$ and $\mathbf{c}_{\mathbf{r}}$ requires $\mathcal{O}(n)G_1$ and $\mathcal{O}(n)G_2$, respectively.

In the Σ_{elim} phase, for each $I \in \{L, R\}$ and $i \in [4]$, computing the inner pairing products X_I, D_{3I}, D'_{iI} requires $\mathcal{O}(n/2)P$. Constructing the updated vectors $\mathbf{b}^*$ and $\mathbf{k}^*$ takes $\mathcal{O}(n)G_2$ and $\mathcal{O}(n)G_1$, respectively. Additionally, computing the folded values $C_{\pm}, X_{\pm}, K_{\pm}$ requires $\mathcal{O}(n/2)P$. The updated vectors $\mathbf{a}', \mathbf{a}_{\mathbf{r}}', \mathbf{k}'$ require $\mathcal{O}(n/2)G_1$, while $\mathbf{b}'$ requires $\mathcal{O}(n/2)G_2$.

Finally, in the Σ_{check} phase, the prover computes the randomized elements E_1 and E_2 , each requiring $\mathcal{O}(1)G_1$ and $\mathcal{O}(1)G_2$, respectively.

Round Complexity and Proof Size. The total number of rounds in the elimination folding process is $\log n$.

In the Σ_{ref} phase, the proof consists of the elements $C, C', X, K, D_3, D_4, X', K', D'_3, D'_4$, contributing $10\mathbb{G}_T$ to the proof size.

During each round of Σ_{elim} , additional proof elements $X_I, D_{3I}, D'_{iI}, C_{\pm}, X_{\pm}, K_{\pm}$ are introduced, contributing $18\mathbb{G}_T$ per round. Since elimination folding is applied for $\log n$ rounds, the total proof size accumulates to $\mathcal{O}(\log n)\mathbb{G}_T$.

In the final Σ_{check} phase, the proof includes the elements E_1 and E_2 , which contribute $\mathcal{O}(1)\mathbb{G}_1$ and $\mathcal{O}(1)\mathbb{G}_2$, respectively.

Verifier Complexity. The verification process primarily involves the Σ_{elim} and Σ_{check} phases.

In each Σ_{elim} step, the verifier computes the values C', X', K', D'_i , requiring $7G_T$ operations. Thus, the total verifier complexity accumulates to $\mathcal{O}(\log n)G_T$.

During Σ_{check} , the verifier performs the computations $E_1 + d \cdot ck_1$ and $E_2 + d^{-1} \cdot ck_2$, requiring $1G_1$ and $1G_2$, respectively. Finally, the verification of the pairing equation involves $1P + \mathcal{O}(1)G_T$ operations.

Summary. Our Π_{HP} achieves a proof size of $\mathcal{O}_{\lambda}(\log n) = \mathcal{O}(\log n)\mathbb{G}_T$ and a sublinear verification complexity of $\mathcal{O}_{\lambda}(\log n) = \mathcal{O}(\log n)G_T$, while maintaining a linear prover cost of $\mathcal{O}_{\lambda}(n) = \mathcal{O}(n)P$. By leveraging precomputations and elimination folding, we significantly reduce the computational burden on the verifier, making the protocol highly scalable for large input sizes.

5 Batching Scheme for Elimination Folding

In this section, we propose a batching technique for the elimination folding algorithm Σ_{elim} . Specifically, we design a method to combine two elimination folding relations into a single relation. In the original Π_{HP} protocol, the prover and verifier must each execute Σ_{elim} $2 \log n$ times and Σ_{check} twice. However, by applying the proposed batching technique, these executions are reduced to just $\log n$ for Σ_{elim} and once for Σ_{check} . Consequently, the computational cost for both the prover and verifier, as well as the proof size in Σ_{elim} and Σ_{check} , is reduced by half.

In Π_{HP} , the prover and verifier first execute Σ_{ref} , obtaining two elimination folding relations:

$$\mathcal{R}_1 = (C, X, K, D_{t \in [4]}; \mathbf{a}, \mathbf{b}, \mathbf{a}_r, \mathbf{k}) \in \mathcal{R}_{r,n,\mathbf{ck}_i}^{\text{Elim}}$$

and

$$\mathcal{R}_2 = (C', X', K', D'_{t \in [4]}; \mathbf{a}', \mathbf{b}', \mathbf{a}'_r, \mathbf{k}') \in \mathcal{R}_{r,n,\mathbf{ck}_i}^{\text{Elim}}.$$

To achieve batching, the prover utilizes a challenge δ to combine the witness vectors $\mathbf{a}$ and $\mathbf{a}'$. Similarly, the prover combines the witness vectors $\mathbf{b}$ and $\mathbf{b}'$, while the verifier exploits the linearity of the inner product operation to verify the result. The structure of our batching algorithm is similar to the approach in Section 3.4 of Dory [22]. As a result, the two separate relations, $\mathcal{R}_1$ and $\mathcal{R}_2$, are combined into a single elimination folding relation, denoted as $\mathcal{R}_{\text{batch}}$.

The detailed batching process is presented in Algorithm 6, and we refer to this batched interactive proof as Σ_{batch} .

Algorithm 6 Σ_{batch} : Batching Hadamard Product

Input : $(\mathcal{R}_1, \mathcal{R}_2) \in (\mathcal{R}_{r,n,\mathbf{ck}_i}^{\text{Elim}})^2$
1: $\mathcal{P} \rightarrow \mathcal{V} : Z_C = \langle \mathbf{a}, \mathbf{b}' \rangle + \langle \mathbf{a}', \mathbf{b} \rangle, \quad Z_X = \langle \mathbf{a}_r, \mathbf{b}' \rangle + \langle \mathbf{a}'_r, \mathbf{b} \rangle, \quad Z_K = \langle \mathbf{k}, \mathbf{b}' \rangle + \langle \mathbf{k}', \mathbf{b} \rangle$
2: $\mathcal{V} \rightarrow \mathcal{P} : \delta \leftarrow \mathbb{F}$
3: $\mathcal{P} : \bar{\mathbf{a}} \leftarrow \delta \mathbf{a} + \mathbf{a}', \quad \bar{\mathbf{b}} \leftarrow \delta \mathbf{b} + \mathbf{b}'$
 $\quad \bar{\mathbf{a}}_r \leftarrow \delta \mathbf{a}_r + \mathbf{a}'_r, \quad \bar{\mathbf{k}} \leftarrow \delta \mathbf{k} + \mathbf{k}'$
4: $\mathcal{V} : \bar{C} \leftarrow \delta^2 C + \delta Z_C + C'$
 $\quad \bar{X} \leftarrow \delta^2 X + \delta Z_X + X'$
 $\quad \bar{K} \leftarrow \delta^2 K + \delta Z_K + K'$
 $\quad \bar{D}_{t \in [4]} \leftarrow \delta D_{t \in [4]} + D'_{t \in [4]}$
Output : $\mathcal{R}_{\text{batch}} = (\bar{C}, \bar{X}, \bar{K}, \bar{D}_{t \in [4]}; \bar{\mathbf{a}}, \bar{\mathbf{b}}, \bar{\mathbf{a}}_r, \bar{\mathbf{k}}) \in \mathcal{R}_{r,n,\mathbf{ck}_i}^{\text{Elim}}$

Theorem 4. *For a given commitment keys $\mathbf{ck}_1 \leftarrow \mathbb{G}_1^n$, and $\mathbf{ck}_2 \leftarrow \mathbb{G}_2^n$, Batch- Σ_{check} is a HVSZK, public-coin, succinct interactive argument of knowledge for $(\mathcal{R}_{r,n,\mathbf{ck}_i}^{\text{Elim}})^2$ with $(3, 3/|\mathbb{F}|)$ -tree extractability under SXDH.*

Proof. Our proof can be found in Appendix D.6. □

After executing Σ_{batch} , the prover and verifier run Σ_{elim} using $\mathcal{R}_{\text{batch}}$ as input. The detailed optimized protocol is presented in Algorithm 7. .

Algorithm 7 Optimized Hadamard Product Arguments

Precompute : for all $0 \leq j \leq m-1$, compute
 $\mathbf{ck}_{1,j+1} = (\mathbf{ck}_{1,j})_L$, $\mathbf{ck}_{2,j+1} = (\mathbf{ck}_{2,j})_L$,
for all $0 \leq j \leq m$, compute
 $\chi_j = \langle \mathbf{ck}_{1,j}, \mathbf{ck}_{2,j} \rangle$,
and for all $0 \leq j \leq m-1$, compute
 $\Delta_{1L,j} = \langle (\mathbf{ck}_{1,j})_L, \mathbf{ck}_{2,j+1} \rangle$, $\Delta_{2L,j} = \langle \mathbf{ck}_{1,j+1}, (\mathbf{ck}_{2,j})_L \rangle$,
 $\Delta_{1R,j} = \langle (\mathbf{ck}_{1,j})_R, \mathbf{ck}_{2,j+1} \rangle$, $\Delta_{2R,j} = \langle \mathbf{ck}_{1,j+1}, (\mathbf{ck}_{2,j})_R \rangle$

Input : $\mathcal{R} = (D_1, D_2, D'_1, D'_2; \mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}) \in \mathcal{R}_{2^m, \mathbf{ck}_{1,0}}^{HP}$

- 1: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{ref}}(\mathcal{R}) \rightarrow (\mathcal{R}_1, \mathcal{R}_2) \in (\mathcal{R}_{r, 2^m, \mathbf{ck}_{1,0}}^{\text{elim}})^2$
- 2: $\mathcal{V} : \text{Check if } X = X'$
- 3: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{batch}}(\mathcal{R}_1, \mathcal{R}_2) \rightarrow \mathcal{R}_{\text{batch}} \in \mathcal{R}_{r, 2^m, \mathbf{ck}_{1,0}}^{\text{elim}}$
- For** $1 \leq j \leq m$:
- 4: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{elim}}(\mathcal{R}_{\text{batch}}) \rightarrow \mathcal{R}'_{\text{batch}} \in \mathcal{R}_{r, 2^{m-j}, \mathbf{ck}_{1,j}}^{\text{elim}}$
- 5: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{check}}(\mathcal{R}'_{\text{batch}}) \rightarrow \text{result}$
- 6: $\mathcal{V} : \text{Accept if } \text{result} = 1$

6 Groth16 Proof Aggregation

In this section, we aggregate n proofs into a $\mathcal{O}(\log n)$ -size proof using the Π_{HP} protocol without any additional trusted setup. Our aggregation protocol, $\Pi_{\text{Groth16.Agg}}$, supports proof aggregation for pairing-based SNARKs. Specifically, we present algorithms supporting proofs of Groth16 zk-SNARKs in this paper.

In Groth16 [18], the verifier uses the proof $\pi = (A, B, C) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$ received from the prover to verify a single equation:

$$e(A, B) = e(g^\alpha, h^\beta) + e\left(\sum_{j=0}^{\ell} S_j^{a_j}, h^\gamma\right) + e(C, h^\delta)$$

where $S_j = g^{\frac{\beta u_j(z) + \alpha v_j(z) + w_j(z)}{\gamma}}$. (For detailed information on Groth16 Construction, please refer to Appendix A). Let $\pi_i = (A_i, B_i, C_i)$, $i \in [n]$, be n Groth16 proofs. For each Groth16 proof, the verifier must compute the following pairing equation using the verification key $\mathbf{vk}_i = (g^\alpha, h^\beta, \{S_{i,j}\}, h^\gamma, h^\delta)$:

$$\forall i \in [n], e(A_i, B_i) = e(g^\alpha, h^\beta) + e\left(\sum_{j=0}^{\ell} S_{i,j}^{a_{i,j}}, h^\gamma\right) + e(C_i, h^\delta).$$

This can be defined as a relation as follows:

Definition 13 (Relation for n Groth16). *The relation for n -length Groth16 proofs π_i with respect to a verification key $\mathbf{vk} = (g^\alpha, h^\beta, \{S_j\}, h^\gamma, h^\delta)$: For all*

$i \in [n]$,

$$\left(\begin{array}{l} \left(\{a_{i,j}, CM(A_i), CM(B_i), CM(C_i)\}; \{\pi_i\} \right) : \\ e(A_i, B_i) = e(g^\alpha, h^\beta) + e\left(\sum_{j=0}^{\ell} S_{i,j}^{a_{i,j}}, h^\gamma\right) + e(C_i, h^\delta) \end{array} \right)$$

Typically, due to the different values of all B_i (or A_i), it is impossible to aggregate them by forming a linear combination using a random challenge. We consider each component as a vector and transform n equations into Hadamard pairing product form to simultaneously prove n equations.

We form a vector composed of n Groth16 proofs $\pi_i = (A_i, B_i, C_i)$ as follows: $\mathbf{A} = (A_1, A_2, \dots, A_n) \in \mathbb{G}_1^n$, $\mathbf{B} = (B_1, B_2, \dots, B_n) \in \mathbb{G}_2^n$, $\mathbf{C} = (C_1, C_2, \dots, C_n) \in \mathbb{G}_1^n$. Thus, the n individual proofs can be combined into a single proof denoted as $\pi = (\mathbf{A}, \mathbf{B}, \mathbf{C})$. We define $\mathbf{P} \in \mathbb{G}_1^n$ as the vector obtained by repeating g^α n times. Similarly, $\mathbf{Q} \in \mathbb{G}_2^n$ is the vector obtained by repeating h^β n times, $\mathbf{H} \in \mathbb{G}_2^n$ is the vector obtained by repeating h^γ n times, and $\mathbf{D} \in \mathbb{G}_2^n$ is the vector obtained by repeating h^δ n times. Let $\mathbf{S}$ be a verification vector defined as

$$\left(\sum_{j=0}^{\ell} S_{i,j}^{a_{i,j}}, \dots, \sum_{j=0}^{\ell} S_{i,j}^{a_{n,j}} \right) \in \mathbb{G}_1^n.$$

Then, these n pairing equations are equivalent to a single Hadamard pairing product equation:

$$\mathbf{A} \circ \mathbf{B} = \mathbf{P} \circ \mathbf{Q} + \mathbf{S} \circ \mathbf{H} + \mathbf{C} \circ \mathbf{D}.$$

Similar to the Hadamard pairing product argument, the above equation can be proven using a random challenge $r \leftarrow \mathbb{F}$, demonstrating that

$$\langle \mathbf{A}_r, \mathbf{B} \rangle = \langle \mathbf{P}_r, \mathbf{Q} \rangle + \langle \mathbf{S}_r, \mathbf{H} \rangle + \langle \mathbf{C}_r, \mathbf{D} \rangle.$$

where

$$\begin{aligned} \mathbf{A}_r &= (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{A}, \\ \mathbf{P}_r &= (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{P}, \\ \mathbf{S}_r &= (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{S}, \\ \mathbf{C}_r &= (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{C}. \end{aligned}$$

In this equation, by utilizing $\mathbf{vk}$, the verifier can compute the term $\langle \mathbf{P}_r, \mathbf{Q} \rangle + \langle \mathbf{S}_r, \mathbf{H} \rangle$. Therefore, similar to the Hadamard pairing product relation, we define the relation in $\Pi_{\text{Gro16.Agg}}$ as follows:

Definition 14 (Relation for $\Pi_{\text{Gro16.Agg}}$). *The relation for $\Pi_{\text{Gro16.Agg}}$ is defined as follows: Let*

- $\mathbf{U} := \{a_{i,j}\}_{0 \leq j \leq \ell}^{1 \leq i \leq n}$ be an instance matrix.
- $\mathbf{vk} := (g^\alpha, h^\beta, S_j, h^\gamma, h^\delta)$ be a verification key.

Then $\mathcal{R}_{n, \mathbf{ck}_1, \mathbf{ck}_2}^{gra}$ with vk is a

$$\left\{ (\mathbf{U}, D_1, D_2, D'_1 ; \mathbf{A}, \mathbf{B}, \mathbf{C}) \left| \begin{array}{l} D_1 = \langle \mathbf{A}, \mathbf{ck}_2 \rangle, \\ D_2 = \langle \mathbf{ck}_1, \mathbf{B} \rangle, \\ D'_1 = \langle \mathbf{C}, \mathbf{ck}_2 \rangle, \\ \mathbf{P} = (g^\alpha, g^\alpha, \dots, g^\alpha), \\ \mathbf{Q} = (h^\beta, h^\beta, \dots, h^\beta), \\ \mathbf{H} = (h^\gamma, h^\gamma, \dots, h^\gamma), \\ \mathbf{D} = (h^\delta, h^\delta, \dots, h^\delta), \\ \mathbf{S} = \left(\sum_{j=0}^{\ell} S_{i,j}^{a_{1,j}}, \dots, \sum_{j=0}^{\ell} S_{i,j}^{a_{n,j}} \right), \\ \mathbf{A} \circ \mathbf{B} = \mathbf{P} \circ \mathbf{Q} + \mathbf{S} \circ \mathbf{H} + \mathbf{C} \circ \mathbf{D} \end{array} \right. \right\}.$$

To verify the above relation, similar to the Hadamard pairing product, reformatting is performed, and additionally, the verifier must compute Φ . To achieve this, the prover generates a vector $\mathbf{D}$ and computes its commitment D'_1 , which is then sent to the verifier. The prover and verifier execute the reformatting algorithm Σ_{ref} using $(D_1, D_2, D'_1, D'_2; \mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$. Additionally, the verifier computes $\Phi = \langle \mathbf{r} \circ \mathbf{P}, \mathbf{Q} \rangle + \langle \mathbf{r} \circ \mathbf{S}, \mathbf{H} \rangle$ using the verification key vk .

Algorithm 8 $\Sigma_{\text{Gro16.Ref}}$: Reformatting for $\Pi_{\text{Gro16.Agg}}$

Input : $(\mathbf{U}, D_1, D_2, D'_1; \mathbf{A}, \mathbf{B}, \mathbf{C})$

- 1: $\mathcal{P} : \mathbf{D} \leftarrow (h^\delta, h^\delta, \dots, h^\delta)$
- 2: $\mathcal{P} \rightarrow \mathcal{V} : D'_2 = \langle \mathbf{ck}_1, \mathbf{D} \rangle$
- 3: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{ref}}(D_1, D_2, D'_1, D'_2; \mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}) \rightarrow (\mathcal{R}_1, \mathcal{R}_2)$
- 4: $\mathcal{V} : \text{Compute } \mathbf{P} = (g^\alpha, g^\alpha, \dots, g^\alpha),$
 $\mathbf{Q} = (h^\beta, h^\beta, \dots, h^\beta),$
 $\mathbf{H} = (h^\gamma, h^\gamma, \dots, h^\gamma),$
 $\mathbf{S} = \left(\sum_{j=0}^{\ell} S_{i,j}^{a_{1,j}}, \dots, \sum_{j=0}^{\ell} S_{i,j}^{a_{n,j}} \right),$
 $\Phi = \langle (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{P}, \mathbf{Q} \rangle + \langle (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{S}, \mathbf{H} \rangle$

Output : $\mathcal{R}_{\text{Groth}} = (\Phi, \mathcal{R}_1, \mathcal{R}_2)$

After the reformatting process, the verifier obtains the instance $(\Phi, C, X, K, D_{t \in [4]}, C', X', K', D'_{t \in [4]})$. Similar to Π_{HP} , the verifier checks $X = X' + \Phi$. This ensures $\mathbf{A} \circ \mathbf{B} = \mathbf{P} \circ \mathbf{Q} + \mathbf{S} \circ \mathbf{H} + \mathbf{C} \circ \mathbf{D}$ with an error probability of $(n-1)/|\mathbb{F}|$. Finally, the verifier verifies if the witness $\mathbf{A}_r$ and $\mathbf{C}_r$ are computed correctly by executing Σ_{elim} and Σ_{check} . With this, we verify all n Groth16 verification equations. Our proof aggregation scheme $\Pi_{\text{Gro16.Agg}}$ does not require a trusted setup and has a verification time of $\mathcal{O}_\lambda(\log n)$. Additionally, it has proof size $\mathcal{O}(\log n)$ and proof generation time $\mathcal{O}_\lambda(n)$, demonstrating its efficiency.

Theorem 5. Let $n = 2^m$. For a commitment keys $\mathbf{ck}_1 \leftarrow \$ \mathbb{G}_1^n$ and $\mathbf{ck}_2 \leftarrow \$ \mathbb{G}_2^n$, the $\Pi_{\text{Gro16.Agg}}$ is a HVSZK, public-coin, succinct interactive argument of knowledge for $\mathcal{R}_{n, \mathbf{vk}, \mathbf{ck}_1}^{\text{AGG}}$ with $(6 \cdot 9^{m+1}, 57 \cdot 9^m / |\mathbb{F}|)$ tree extractability under SXDH.

Algorithm 9 $\Pi_{\text{Gro16.Agg}}$: Groth16 proof aggregation

Input : $(\mathbf{U}, D_1, D_2, D'_1; \mathbf{A}, \mathbf{B}, \mathbf{C})$

- 1: $\mathcal{P}, \mathcal{V}$: Run $\Sigma_{\text{Gro16.Ref}}(\mathbf{U}, D_1, D_2, D'_1; \mathbf{A}, \mathbf{B}, \mathbf{C}) \rightarrow \mathcal{R}_{\text{Groth}}$
- 2: $\mathcal{V}$: Check if $X = X' + \Phi$
- 3: $\mathcal{P}, \mathcal{V}$: Run $\Sigma_{\text{batch}}(\mathcal{R}_1, \mathcal{R}_2) \rightarrow \mathcal{R}_{\text{batch}}$
- 4: $\mathcal{P}, \mathcal{V}$: Run $\Sigma_{\text{elim}}(\mathcal{R}_{\text{batch}}) \rightarrow \mathcal{R}'_{\text{batch}}$
- 5: $\mathcal{P}, \mathcal{V}$: Run $\Sigma_{\text{check}}(\mathcal{R}'_{\text{batch}}) \rightarrow \text{result}$
- 6: $\mathcal{V}$: Accept if $\text{result} = 1$

Proof. The proof is straightforward by Theorem 3. □

Table 3: Comparison of asymptotic performance of Groth16 aggregation. Here, n represents the number of Groth16 proofs.

Protocol	additional trusted setup	$\mathcal{P}$'s time	Proof size	$\mathcal{V}$'s time	CRS size	Assumption
SnarkPack [15]	✓	$\mathcal{O}_\lambda(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}_\lambda(\log n)$	$\mathcal{O}(n)$	ASDGH, GGM
$\Pi_{\text{Gro16.Agg}}$	✗	$\mathcal{O}_\lambda(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}_\lambda(\log n)$	$\mathcal{O}(n)$	SXDH

7 KZG Proof Aggregation

In this section, we introduce the $\Pi_{\text{KZG.Agg}}$ for aggregating KZG commitments using our ZKP Π_{HP} . Our scheme $\Pi_{\text{KZG.Agg}}$ aims to minimize verification costs by reducing the pairing operations performed by the Verifier to a constant size.

In the KZG commitment scheme, the verifier computes the pairing equation for verification:

$$e(C - g^e, h) = e(\pi, h^{z-s})$$

We explore the generation of KZG proofs π_i with random challenges $s_i \leftarrow \mathbb{F}$ in the general KZG commitment scheme. Additionally, we consider the scenario where all polynomials are committed using different commitment keys (the same s_i , $f_i(x)$, or commitment keys' usage can be considered for specific cases). Consequently, in our setting, the verifier needs to compute the following n pairing equations:

$$\begin{aligned}
e(C_1 - g^{e_1}, h) &= e(\pi_1, h^{z_1 - s_1}), \\
e(C_2 - g^{e_2}, h) &= e(\pi_2, h^{z_2 - s_2}), \\
&\vdots \\
e(C_n - g^{e_n}, h) &= e(\pi_n, h^{z_n - s_n}).
\end{aligned}$$

(In this section, we assume that all polynomials have the same degree.) Since the left side uses the same h , we can compress the single pairing equation by computing a linear combination $C = \sum_{i=1}^n r^{i-1}(C_i - g^{e_i})$ using challenge $r \leftarrow \mathbb{F}$:

$$e(C, h) = e(\pi_1, h^{z_1-s_1}) \cdot e(\pi_2, h^{z_2-s_2})^r \cdots e(\pi_n, h^{z_n-s_n})^{r^{n-1}}$$

However, this still requires $\mathcal{O}(n)$ pairing operations. Therefore, to efficiently prove n KZG commitments, we employ a Hadamard product argument Π_{HP} . In our setting, the n pairing equations are represented in a Hadamard product form $\mathbf{a} \circ \mathbf{b} = \mathbf{c} \circ \mathbf{d}$, utilizing only the commitments of each vector, allowing verification of all KZG commitments with just one pairing operation where $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$, $\mathbf{d}$ are defined as follows.

$$\begin{aligned} \mathbf{a} &:= (C_1 - g^{e_1}, C_2 - g^{e_2}, \dots, C_n - g^{e_n}) \in \mathbb{G}_1^n \\ \mathbf{b} &:= (h, h, \dots, h) \in \mathbb{G}_2^n \\ \mathbf{c} &:= (\pi_1, \pi_2, \dots, \pi_n) \in \mathbb{G}_1^n \\ \mathbf{d} &:= (h^{z_1-s_1}, h^{z_2-s_2}, \dots, h^{z_n-s_n}) \in \mathbb{G}_2^n \end{aligned}$$

Our goal is to verifier-efficiently compute commitments for each vector $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$, $\mathbf{d}$. Each vector is committed using commitment keys $\mathbf{ck}_1$ and $\mathbf{ck}_2$ along with AFGHO commitments. Specifically:

$$\begin{aligned} \text{CM}(\mathbf{ck}_2; \mathbf{a}) &= \langle \mathbf{a}, \mathbf{ck}_2 \rangle, \quad \text{CM}(\mathbf{ck}_1; \mathbf{b}) = \langle \mathbf{ck}_1, \mathbf{b} \rangle, \\ \text{CM}(\mathbf{ck}_2; \mathbf{c}) &= \langle \mathbf{c}, \mathbf{ck}_2 \rangle, \quad \text{CM}(\mathbf{ck}_1; \mathbf{d}) = \langle \mathbf{ck}_1, \mathbf{d} \rangle \end{aligned}$$

If the verifier computes commitments directly, each commitment requires n pairing operations. To reduce this, we utilize properties of pairings and the doubly homomorphic property of AFGHO commitments.

1. For $\text{CM}(\mathbf{ck}_2; \mathbf{a})$, we have the reduced equation

$$\langle (C_1, C_2, \dots, C_n), \mathbf{ck}_2 \rangle = e(g, \prod ck_{2j}^{e_j}).$$

Thus, the verifier can compute $e(g, \prod ck_{2j}^{e_j})$ using 1 pairing operation and n group exponentiation operations, and then use the received $X = \langle (C_1, C_2, \dots, C_n), \mathbf{ck}_2 \rangle$ from the prover to compute $\text{CM}(\mathbf{ck}_2; \mathbf{a})$.

2. Since h is public, the verifier can precompute the commitment $\text{CM}(\mathbf{ck}_1; \mathbf{b}) = \langle \mathbf{ck}_1, (h, h, \dots, h) \rangle$.
3. The commitment $\text{CM}(\mathbf{ck}_2; \mathbf{c}) = \langle (\pi_1, \pi_2, \dots, \pi_n), \mathbf{ck}_2 \rangle$ serves as the commitment for the proof, which the verifier can obtain from the prover without any additional computation.
4. Lastly, $\text{CM}(\mathbf{ck}_1; \mathbf{d})$ follows a similar reduced equation as $\text{CM}(\mathbf{ck}_2; \mathbf{a})$:

$$\langle \mathbf{ck}_1, (h^{z_1}, h^{z_2}, \dots, h^{z_n}) \rangle = e\left(\prod ck_{1j}^{s_j}, h\right)$$

In this equation, the first term can be precomputed by the verifier, and the second term can be computed with just one pairing operation. Therefore, the verifier can efficiently compute $\text{CM}(\mathbf{ck}_1; \mathbf{d})$.

Algorithm 10 $\Sigma_{\text{Aggregator}}$: KZG Aggregator

Precompute: $D_2 = \langle \mathbf{ck}_1, (h, h, \dots, h) \rangle$,
 $Y = \langle \mathbf{ck}_1, (h^{z_1}, h^{z_2}, \dots, h^{z_n}) \rangle$

- 1: $\mathcal{P}$: For all $i \in [n]$,
 $C_i \leftarrow \text{cm}(f_i(x); \mathbf{pk}_i)$
and $\mathbf{cm} \leftarrow (C_1, C_2, \dots, C_n)$
- 2: $\mathcal{P} \rightarrow \mathcal{V} : X = \langle \mathbf{cm}, \mathbf{ck}_2 \rangle$
- 3: $\mathcal{V} \rightarrow \mathcal{P} : s_1, \dots, s_n \leftarrow_{\$} \mathbb{F}$
- 4: $\mathcal{P} \rightarrow \mathcal{V} : \text{For all } i \in [n], \quad e_i \leftarrow f_i(s_i)$
- 5: $\mathcal{P} : \text{For all } i \in [n]$,
 $\pi_i \leftarrow g^{\frac{f_i(z_i) - e_i}{z_i - s_i}}$ and $\boldsymbol{\pi} \leftarrow (\pi_1, \pi_2, \dots, \pi_n)$
- 6: $\mathcal{P} \rightarrow \mathcal{V} : D'_1 = \langle \boldsymbol{\pi}, \mathbf{ck}_2 \rangle$
- 7: $\mathcal{V} : \mathbf{e} \leftarrow (e_1, e_2, \dots, e_n), \quad \mathbf{s} \leftarrow (s_1, s_2, \dots, s_n)$
- 8: $\mathcal{V} : D_1 = X + e(g, \mathbf{ck}_2^{\mathbf{e}}), \quad D'_2 = Y + e(\mathbf{ck}_1^{\mathbf{s}}, h)$

Output: $\mathcal{R}_{KZG} = (D_1, D_2, D'_1, D'_2; \mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d})$

Prover and verifier execute the KZG Aggregator to obtain the relation $(D_1, D_2, D'_1, D'_2; \mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}) \in \mathcal{R}_{n, \mathbf{ck}_1, \mathbf{ck}_2}^{HP}$. This enables the use of Π_{HP} , and our KZG proof aggregation scheme $\Pi_{\text{KZG.Agg}}$ is completed by executing KZG Aggregator and Π_{HP} .

Algorithm 11 $\Pi_{\text{KZG.Agg}}$: KZG Proof Aggregation

- 1: $\mathcal{P}, \mathcal{V} : \Sigma_{\text{Aggregator}}(\text{srs}; \{f_i(x)\}) \rightarrow \mathcal{R}_{KZG}$
- 2: $\mathcal{P}, \mathcal{V} : \Pi_{\text{HP}}(\mathcal{R}_{KZG}) \rightarrow \text{result}$
- 3: $\mathcal{V} : \text{Accept if } \text{result} = 1$

7.1 Analysis

Generally, n KZG aggregations require the verifier to perform $2n$ pairing operations, resulting in high verification costs. In this subsection, we compare and analyze the performance of KZG commitment aggregation schemes [14], [3], and our $\Pi_{\text{KZG.Agg}}$ to address this drawback.

Table 4 shows the opening performance of each protocol for n distinct polynomials $f_1(x), f_2(x), \dots, f_n(x)$ and points $s_1, s_2, \dots, s_n$ (we consider only cases where all polynomials have the same degree d). [14] proposed a batch KZG scheme that allows efficient verification for n polynomials $f_i(x)$. However, it is efficient only when the openings are for the same point s , and the verification cost increases with the number of distinct points s_i . In this case, the proof size is $\mathcal{O}(dn)$, and the verifier needs $\mathcal{O}(n)$ group multiplications. [3] improved on this by enabling efficient aggregation of openings for f_i and n distinct points s_i . [3] reduced the proof size to a constant and decreased the prover's cost from $\mathcal{O}(dn)$ to $\mathcal{O}(d)$, making it independent of the number of openings. Our aggregation scheme considers a more general case, supporting KZG opening aggregation where the

polynomials, points, and **srs** are all distinct. Our $\Pi_{\text{KZG.Agg}}$ has a proof size of $\mathcal{O}(\log n)$ for the general case and can be verified with $\mathcal{O}(\log n)\mathbb{G}_T$ operations and only 3 pairing operations. Compared to the two aggregation schemes above, our results reduce the verifier’s cost from $\mathcal{O}(n)\mathbb{G}_1$ to $\mathcal{O}(\log n)\mathbb{G}_T$. This means it is more efficient in terms of verification, requiring only 3 pairing operations, which is comparable to the operations needed by the previous schemes. Additionally, our scheme effectively supports KZG aggregation even when all **srs** are distinct, in which case the size of the **srs** increases proportionally to n as $\mathcal{O}(dn)$.

Table 4: Comparison of opening complexity of KZG aggregation

	[14]	[3]	$\Pi_{\text{KZG.Agg}}$
$\mathcal{P}$ ’s work	$\mathcal{O}(dn)\mathbb{G}_1$	$\mathcal{O}(d)\mathbb{G}_1$	$\mathcal{O}(n)P, \mathcal{O}(dn)\mathbb{G}_1$
Proof size	$\mathcal{O}(n)\mathbb{G}_1$	$\mathcal{O}(1)\mathbb{G}_1$	$\mathcal{O}(\log n)\mathbb{G}_T$
$\mathcal{V}$ ’s work	$\mathcal{O}(n)\mathbb{G}_1, 2P$	$\mathcal{O}(n)\mathbb{G}_1, 2P$	$\mathcal{O}(\log n)\mathbb{G}_T, 3P$
Support for distinct polynomials	✓	✓	✓
Support for distinct points	✗	✓	✓
Support for distinct srs	✗	✗	✓
srs size (distinct srs case)	$\mathcal{O}(d)\mathbb{G}_1$ (✗)	$\mathcal{O}(d)\mathbb{G}_1$ (✗)	$\mathcal{O}(d)\mathbb{G}_1$ $(\mathcal{O}(dn)\mathbb{G}_1)$

8 Comparison and Evaluation

In this section, we compare the Hadamard product argument techniques of Bulletproofs [7] and Bünz et al. [8] with our proposed Hadamard Product Argument (Π_{HP}). Additionally, we analyze the performance differences between the optimized verifier variant, $\text{opt-}\Pi_{\text{HP}}$, and the original Π_{HP} . Furthermore, we evaluate the efficiency of our Π_{HP} -based aggregation method by comparing it with SnarkPack [15], a conventional Groth16 [18] proof aggregation scheme, focusing on proof generation and verification times.

For implementation, we developed Π_{HP} using Rust, leveraging the arkworks[10] library and the BLS-381 curve to optimize cryptographic operations. All experiments were conducted in a macOS environment on an Apple M1 Pro with 32GB of memory. To ensure a fair performance comparison, all baseline techniques, including Bulletproofs, Bünz et al., and SnarkPack, were executed under identical conditions. The experiments measured proof generation and verification times to assess the performance improvements of Π_{HP} and $\text{opt-}\Pi_{\text{HP}}$ over existing methods.

8.1 HPA

Figure 2 presents a benchmark comparing our proposed Π_{HP} against Bulletproofs and Bünz et al. While all three methods exhibit linear prover complexity, their

verifier complexities differ. Bulletproofs operates with linear verifier complexity, whereas both Bünz et al. and our Π_{HP} achieve logarithmic verifier complexity. However, Bünz et al. requires a trusted setup.

For vector sizes exceeding 2^{13} , our Π_{HP} significantly outperforms Bulletproofs in verification efficiency. If prover efficiency is the primary concern, Bulletproofs remains the best choice. However, for applications demanding both efficient proof generation and verification, our Π_{HP} emerges as the optimal solution.

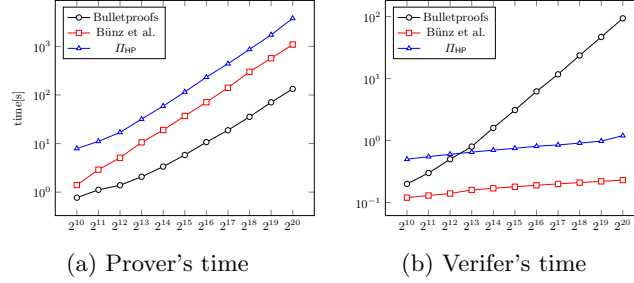


Fig. 2: Performance of Hadamard product arguments varying vector length

8.2 Batch HPA

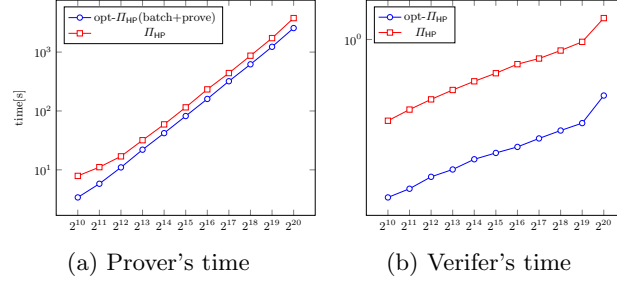
Figure 3 compares the performance of opt- Π_{HP} against the original Π_{HP} . While the original Π_{HP} required executing the Elimination Folding Algorithm twice during verification, opt- Π_{HP} optimizes this process by batching them into a single step, effectively halving the verifier's computational workload.

As shown in Figure 3(b), this optimization results in approximately twice the verification speed compared to the original Π_{HP} . The impact becomes even more significant as the input size grows, reducing computational overhead for large-scale verification.

On the other hand, as depicted in Figure 3(a), proof generation times remain similar for both opt- Π_{HP} and the original Π_{HP} . This indicates that the optimization reduces verifier computation without increasing the prover's burden, making it particularly effective for real-world applications where verifier efficiency is a primary concern.

8.3 Aggregation for Groth16

We compare the efficiency of our aggregation scheme against SnarkPack. Both schemes share the same asymptotic complexity ($\mathcal{O}_\lambda(n)$ proving time, $\mathcal{O}_\lambda(\log n)$ verification time, and $\mathcal{O}(\log n)$ proof size), but our approach offers stronger security due to its transparency. Table 3 provides an asymptotic performance comparison between SnarkPack and our scheme.

Fig. 3: Performance comparison of $\text{opt-}\Pi_{HP}$ and Π_{HP}

Both protocols are pairing-based. SnarkPack relies on the ASDGH (Auxiliary Structured Double Group Pairing) assumption, which requires an additional GGM (Generic Group Model). In contrast, our $\Pi_{Gro16.Agg}$ achieves soundness solely under the SXDH assumption, eliminating the need for the GGM model.

Figure 4 shows that both schemes exhibit comparable proof generation times. Although evaluations beyond 2^{18} proofs were not conducted on a 32GB RAM M1 Pro MacBook, extrapolation suggests similar performance trends at larger scales. While SnarkPack achieves faster verification times, our approach remains practically efficient. Thus, in applications where both security and practical efficiency are crucial, our scheme provides a more robust and transparent alternative.

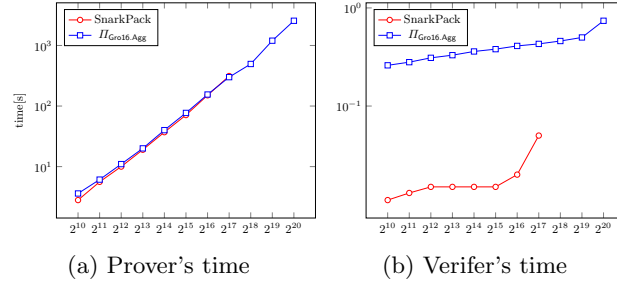


Fig. 4: Performance of Groth16 aggregation varying number of proof

8.4 Code Availability

The source code for the implementation presented in this section (Section 8) is publicly available at <https://github.com/5Donghwan/HPA>.

References

1. Abe, M., Fuchsbauer, G., Groth, J., Haralambiev, K., Ohkubo, M.: Structure-preserving signatures and commitments to group elements. In: Annual Cryptology Conference. pp. 209–236. Springer (2010)
2. Bayer, S., Groth, J.: Efficient zero-knowledge argument for correctness of a shuffle. In: Advances in Cryptology–EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15–19, 2012. Proceedings 31. pp. 263–280. Springer (2012)
3. Boneh, D., Drake, J., Fisch, B., Gabizon, A.: Efficient polynomial commitment schemes for multiple points and polynomials. Cryptology ePrint Archive (2020)
4. Boneh, D., Drijvers, M., Neven, G.: Compact multi-signatures for smaller blockchains. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 435–464. Springer (2018)
5. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 327–357. Springer (2016)
6. Bootle, J., Chiesa, A., Groth, J.: Linear-time arguments with sublinear verification from tensor codes. In: Theory of Cryptography: 18th International Conference, TCC 2020, Durham, NC, USA, November 16–19, 2020. Springer (2020)
7. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE symposium on security and privacy (SP). pp. 315–334. IEEE (2018)
8. Bünz, B., Maller, M., Mishra, P., Tyagi, N., Vesely, P.: Proofs for inner pairing products and applications. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 65–97. Springer (2021)
9. Campanelli, M., Fiore, D., Querol, A.: Legosnark: Modular design and composition of succinct zero-knowledge proofs. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 2075–2092 (2019)
10. arkworks contributors: `arkworks` zkSNARK ecosystem (2022), <https://arkworks.rs>
11. Daza, V., Ràfols, C., Zacharakis, A.: Updateable inner product argument with logarithmic verifier and applications. In: Public-Key Cryptography–PKC 2020: 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography, Edinburgh, UK, May 4–7, 2020, Proceedings, Part I 23. pp. 527–557. Springer (2020)
12. Dellepère, M., Mishra, P., Shirzad, A.: Garuda and pari: Faster and smaller snarks via efficient polynomial commitments. Cryptology ePrint Archive (2024)
13. Fauzi, P., Lipmaa, H., Zhang, B.: Efficient modular NIZK arguments from shift and product. In: Cryptology and Network Security: 12th International Conference, CANS 2013, Paraty, Brazil. Springer (2013)
14. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
15. Gailly, N., Maller, M., Nitulescu, A.: Snarkpack: practical snark aggregation. In: Financial Cryptography and Data Security: 26th International Conference, FC 2022, Grenada, May 2–6, 2022, Revised Selected Papers. Springer (2022)
16. Groth, J.: Linear algebra with sub-linear zero-knowledge arguments. In: Advances in Cryptology–CRYPTO 2009: 29th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 16–20, 2009. Proceedings. Springer (2009)

17. Groth, J.: Short pairing-based non-interactive zero-knowledge arguments. In: *Asiacrypt*. vol. 6477, pp. 321–340. Springer (2010)
18. Groth, J.: On the size of pairing-based non-interactive arguments. In: *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Vienna, Austria, May 8–12, 2016, Proceedings, Part II 35. pp. 305–326. Springer (2016)
19. Groth, J., Ishai, Y.: Sub-linear zero-knowledge argument for correctness of a shuffle. In: *Advances in Cryptology–EUROCRYPT 2008: 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Istanbul, Turkey, April 13–17, 2008. Proceedings 27. pp. 379–396. Springer (2008)
20. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security*, Singapore, December 5–9, 2010. Proceedings 16. pp. 177–194. Springer (2010)
21. Kothapalli, A., Masserova, E., Parno, B.: Poppins: a direct construction for asymptotically optimal zksnarks. *Cryptology ePrint Archive* (2020)
22. Lee, J.: Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In: *TCC 2021*. Springer (2021)
23. Lipmaa, H.: Progression-free sets and sublinear pairing-based non-interactive zero-knowledge arguments. In: *Theory of Cryptography: 9th Theory of Cryptography Conference, TCC 2012, Taormina, Sicily, Italy, March 19–21, 2012*. Springer (2012)
24. Lipmaa, H., Siim, J., Zajac, M.: Counting vampires: from univariate sumcheck to updatable zk-snark. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 249–278. Springer (2022)
25. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: *Annual international cryptology conference*. Springer (1992)
26. Setty, S.: Spartan: Efficient and general-purpose zksnarks without trusted setup. In: *Annual International Cryptology Conference*. pp. 704–737. Springer (2020)
27. Wahby, R.S., Tzialla, I., Shelat, A., Thaler, J., Walfish, M.: Doubly-efficient zk-snarks without trusted setup. In: *2018 IEEE Symposium on Security and Privacy (SP)*. pp. 926–943. IEEE (2018)
28. Wang, W., Zhou, L., Yaish, A., Zhang, F., Fisch, B., Livshits, B.: Proo ϕ : A zkp market mechanism. *arXiv preprint arXiv:2404.06495* (2024)
29. Xie, J., Hu, Y., Yu, Y.: Hadamard product argument from lagrange-based univariate polynomials. In: *Australasian Conference on Information Security and Privacy*. pp. 472–492. Springer (2024)
30. Zhang, Y., Szepieniec, A., Zhang, R., Sun, S.F., Wang, G., Gu, D.: Voproof: efficient zksnarks from vector oracle compilers. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. pp. 3195–3208 (2022)

A Groth16: Non-interactive linear proofs for quadratic arithmetic programs

In this section, we introduce Groth16 [18], a pairing-based Non-Interactive Zero-Knowledge (NIZK) argument. Groth16 is a zk-SNARK for quadratic arithmetic programs, characterized by constant-sized proofs and proofs being group elements. A quadratic arithmetic program defines the subsequent binary relation, setting $a_0 = 1$:

$$\mathcal{R} = \left\{ (\mathbb{x}, \mathbb{w}) \left| \begin{array}{l} \mathbb{x} = (a_1, a_2, \dots, a_\ell) \wedge \mathbb{w} = (a_{\ell+1}, a_{\ell+2}, \dots, a_m) \\ \wedge \sum_{i=0}^m a_i u_i(x) \cdot \sum_{i=0}^m a_i v_i(x) \equiv \sum_{i=0}^m a_i w_i(x) \pmod{t(x)} \end{array} \right. \right\}$$

Groth.Setup($1^\lambda, \mathcal{R}$)	
$\alpha, \beta, \gamma, \delta \leftarrow \mathbb{Z}_p^*, z \leftarrow \mathbb{Z}_p^*,$ $\sigma_1 = \left(g^\alpha, g^\beta, g^\delta, \left\{ g^{z^i} \right\}_{i=0}^{n-1}, \left\{ g^{\frac{\beta u_i(z) + \alpha v_i(z) + w_i(z)}{\gamma}} \right\}_{i=0}^\ell, \left\{ g^{\frac{\beta u_i(z) + \alpha v_i(z) + w_i(z)}{\delta}} \right\}_{i=\ell+1}^m, \left\{ g^{\frac{z^i t(z)}{\delta}} \right\}_{i=0}^{n-2} \right)$ $\sigma_2 = \left(h^\beta, h^\gamma, h^\delta, \left\{ h^{z^i} \right\}_{i=0}^{n-1} \right)$ $\text{crs} = (\text{QAP}, (\sigma_1, \sigma_2))$ $\text{vk} = \left(P = g^\alpha, Q = h^\beta, \left\{ S_i = g^{\frac{\beta u_i(z) + \alpha v_i(z) + w_i(z)}{\gamma}} \right\}_{i=0}^\ell, H = h^\gamma, D = h^\delta \right)$ $\text{td} = (z, \alpha, \beta, \gamma, \delta)$ return (crs, td)	
Groth.Prove(crs, $\mathbb{x}, \mathbb{w}$)	Groth.Verify(vk, $\mathbb{x}, \pi$)
$\mathbb{x} = (a_1, \dots, a_\ell), a_0 = 1$ $\mathbb{w} = (a_{\ell+1}, \dots, a_m)$ $u(x) = \sum_{i=0}^m a_i u_i(x)$ $u_w(x) = \sum_{i=\ell+1}^m a_i u_i(x)$ $v(x) = \sum_{i=0}^m a_i v_i(x)$ $v_w(x) = \sum_{i=\ell+1}^m a_i v_i(x)$ $w(x) = \sum_{i=0}^m a_i w_i(x)$ $w_w(x) = \sum_{i=\ell+1}^m a_i w_i(x)$ $h(x) = \frac{u(x)v(x) - w(x)}{t(x)}$ $r, s \leftarrow \mathbb{Z}_p^*$ $f_w = \frac{\beta u_w(z) + \alpha v_w(z) + w_w(z)}{\delta}$ $a = \alpha + u(z) + r\delta$ $b = \beta + v(z) + s\delta$ $c = f_w + \frac{t(z)h(z)}{\delta} + sa + rb - sr\delta$ $A = g^a, B = h^b, C = g^c$ return $\pi = (A, B, C)$	$\pi = (A, B, C)$ $u_x(x) = \sum_{i=0}^\ell a_i u_i(x)$ $v_x(x) = \sum_{i=0}^\ell a_i v_i(x)$ $w_x(x) = \sum_{i=0}^\ell a_i w_i(x)$ $f_x = \frac{\beta u_x(z) + \alpha v_x(z) + w_x(z)}{\gamma}$ check : $e(A, B) = e(P, Q) \cdot e(g^{f_x}, H) \cdot e(C, D)$ Groth.Sim(td, $\mathbb{x}$) $a, b \leftarrow \mathbb{Z}_p^*$ $A = g^a, B = h^b$ $c = \frac{ab - \alpha\beta - \beta u_x(z) - \alpha v_x(z) - w_x(z)}{\delta}$ $C = g^c$ return $\pi = (A, B, C)$

Fig. 5: Groth16: a zk-SNARK for QAP

B KZG Polynomial Commitment Scheme

Let $\text{para} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g, h, z)$ be a system parameters. The KZG polynomial commitment scheme [20] defined as follows:

$\Pi_{\text{KZG.Agg}}.\text{Setup}(\text{para})$	$\Pi_{\text{KZG.Agg}}.\text{Commit}(\text{srs}, f(x))$
$\text{srs} = (g^{z^i}, h^{z^i})$	$C = g^{f(z)}$
return srs	return C
$\Pi_{\text{KZG.Agg}}.\text{Open}(\text{srs}, f(x), s)$	
$q(x) = \frac{f(x) - f(s)}{x - s}$	
$a = f(s)$	
$\pi = g^{q(z)}$	
return (a, π)	
$\Pi_{\text{KZG.Agg}}.\text{Verify}(\text{srs}, C, s, a, \pi)$	
check : $e(C - g^a, h) = e(\pi, h^{z-s})$	
return 1/0	

Fig. 6: KZG Polynomial Commitment Scheme

C Dory: An inner product argument with a logarithmic Verifier

In this section, we introduce Dory [22], an inner product arguments with sub-linear verification time. Dory exploits the properties of AFGHO commitments to achieve efficient inner product arguments with verification time reduced to $\mathcal{O}_\lambda(\log n)$.

Dory consists of two main sub-protocols. The first sub-protocol, the Reduce protocol, reduces the length of the given witness vectors $\mathbf{a} \in \mathbb{G}_1^n$ and $\mathbf{b} \in \mathbb{G}_2^n$ by half. The prover and verifier interactively reduce the length of the vectors using commitments. After a logarithmic number of iterations, the vector size becomes 1, and the Reduce step terminates. Subsequently, in the second sub-protocol, the Scalar-Product protocol, the verifier confirms correct computation through pairing operations.

We introduce Dory's Language, Reduce protocol, and Scalar-Product protocol sequentially.

C.1 Language for Dory

Definition 15. *The Language for Dory defined as follows:*

$$\begin{aligned}
(C, D_1, D_2) &\in \mathcal{L}_{n, \mathbf{ck}_1, \mathbf{ck}_2, H_1, H_2} \subset \mathbb{G}_T^3 \\
&\iff \exists (\mathbf{a} \in \mathbb{G}_1^n, \mathbf{b} \in \mathbb{G}_2^n, r_C, r_{D_1}, r_{D_2} \in \mathbb{F}) : \\
&\quad C = \langle \mathbf{a}, \mathbf{b} \rangle + r_C \cdot e(H_1, H_2), \\
&\quad D_1 = \langle \mathbf{a}, \mathbf{ck}_2 \rangle + r_{D_1} \cdot e(H_1, H_2), \quad D_2 = \langle \mathbf{ck}_1, \mathbf{b} \rangle + r_{D_2} \cdot e(H_1, H_2)
\end{aligned}$$

C.2 Dory-Reduce

Algorithm 12 Dory-Reduce

Precompute : $H_T = e(H_1, H_2)$, $\chi = \langle \mathbf{ck}_1, \mathbf{ck}_2 \rangle$
 $\Delta_{1L} = \langle \mathbf{ck}_{1L}, \mathbf{ck}_2' \rangle$, $\Delta_{1R} = \langle \mathbf{ck}_{1R}, \mathbf{ck}_2' \rangle$,
 $\Delta_{2L} = \langle \mathbf{ck}_1', \mathbf{ck}_{2L} \rangle$, $\Delta_{2R} = \langle \mathbf{ck}_1', \mathbf{ck}_{2R} \rangle$,
witness : $(\mathbf{a}, \mathbf{b}, r_C, r_{D_1}, r_{D_2}) \in \mathcal{L}_{2^m, \mathbf{ck}_1, \mathbf{ck}_2, H_1, H_2}$
 $\mathcal{P} : r_{D_{1L}}, r_{D_{1R}}, r_{D_{2L}}, r_{D_{2R}} \leftarrow \mathbb{F}$
 $\mathcal{P} \rightarrow \mathcal{V} : D_{1L} = \langle \mathbf{a}_L, \mathbf{ck}_2' \rangle + r_{D_{1L}} H_T$, $D_{1R} = \langle \mathbf{a}_R, \mathbf{ck}_2' \rangle + r_{D_{1R}} H_T$,
 $D_{2L} = \langle \mathbf{ck}_1', \mathbf{b}_L \rangle + r_{D_{2L}} H_T$, $D_{2R} = \langle \mathbf{ck}_1', \mathbf{b}_R \rangle + r_{D_{2R}} H_T$
 $\mathcal{V} \rightarrow \mathcal{P} : \beta \leftarrow \mathbb{F}$
 $\mathcal{P} : \mathbf{a}^* \leftarrow \mathbf{a} + \beta \mathbf{ck}_1$, $\mathbf{b}^* \leftarrow \mathbf{b} + \beta^{-1} \mathbf{ck}_2$, $r_C \leftarrow r_C + \beta r_{D_2} + \beta^{-1} r_{D_1}$
 $\mathcal{P} : r_{C_+}, r_{C_-} \leftarrow \mathbb{F}$
 $\mathcal{P} \rightarrow \mathcal{V} : C_+ = \langle \mathbf{a}_L, \mathbf{b}_R \rangle + r_{C_+} H_T$, $C_- = \langle \mathbf{a}_R, \mathbf{b}_L \rangle + r_{C_-} H_T$
 $\mathcal{V} \rightarrow \mathcal{P} : \alpha \leftarrow \mathbb{F}$
 $\mathcal{P} : \mathbf{a}' \leftarrow \alpha \mathbf{a}_L^* + \mathbf{a}_R^*$, $\mathbf{b}' \leftarrow \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^*$,
 $r_{D_1}' \leftarrow \alpha r_{D_{1L}} + r_{D_{1R}}$, $r_{D_2}' \leftarrow \alpha^{-1} r_{D_{2L}} + r_{D_{2R}}$, $r_C' \leftarrow r_C + \alpha r_{C_+} + \alpha^{-1} r_{C_-}$
 $\mathcal{V} : C' \leftarrow C + \chi + \beta D_2 + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_-$
 $D_1' \leftarrow \alpha D_{1L} + D_{1R} + \alpha \beta \Delta_{1L} + \beta \Delta_{1R}$
 $D_2' \leftarrow \alpha^{-1} D_{2L} + D_{2R} + \alpha^{-1} \beta^{-1} \Delta_{2L} + \beta^{-1} \Delta_{2R}$
 $\mathcal{V} : \text{Accept if } (C', D_1', D_2') \in \mathcal{L}_{2^{m-1}, \mathbf{ck}_1', \mathbf{ck}_2', H_1, H_2}$
witness : $(\mathbf{a}', \mathbf{b}', r_C', r_{D_1}', r_{D_2}')$

C.3 Scalar-Product

Algorithm 13 Scalar-Product

Precompute : $H_T = e(H_1, H_2)$, $\chi = \langle \mathbf{ck}_1, \mathbf{ck}_2 \rangle$
witness : $(a, b, r_C, r_{D_1}, r_{D_2}) \in \mathcal{L}_{1, \mathbf{ck}_1, \mathbf{ck}_2, H_1, H_2}$
 $\mathcal{P} : r_{P_1}, r_{P_2}, r_Q, r_R \leftarrow \mathbb{F}$, $d_1 \leftarrow \mathbb{G}_1$, $d_2 \leftarrow \mathbb{G}_2$
 $\mathcal{P} \rightarrow \mathcal{V} : P_1 = e(d_1, \mathbf{ck}_2) + r_{P_1} H_T$, $P_2 = e(\mathbf{ck}_1, d_2) + r_{P_2} H_T$,
 $Q = e(d_1, b) + e(a, d_2) + r_Q H_T$, $R = e(d_1, d_2) + r_R H_T$
 $\mathcal{V} \rightarrow \mathcal{P} : c \leftarrow \mathbb{F}$
 $\mathcal{P} \rightarrow \mathcal{V} : E_1 \leftarrow d_1 + ca$, $E_2 \leftarrow d_2 + cb$,
 $r_1 \leftarrow r_{P_1} + cr_{D_1}$, $r_2 \leftarrow r_{P_2} + cr_{D_2}$, $r_3 \leftarrow r_R + cr_Q + c^2 r_C$
 $\mathcal{V} : d \leftarrow \mathbb{F}$, accept if:
 $e(E_1 + d \cdot \mathbf{ck}_1, E_2 + d^{-1} \cdot \mathbf{ck}_2)$
 $= \chi + R + cQ + c^2 C + dP_2 + cdD_2 + d^{-1} P_1 + cd^{-1} D_1 - (r_3 + dr_2 + d^{-1} r_1) H_T$

D Zero-Knowledge Arguments

In this section, we introduce the zero-knowledge arguments of our protocols omitted in the main text. The blue color indicates the zero-knowledge. These include the relation for Half-Hadamard product arguments, reformatting algorithm, final check algorithm, and batching scheme, presented in sequence. The remaining protocol, Π_{HP} , can be expressed as combinations of the zero-knowledge arguments introduced above, so they are omitted.

D.1 Relation for Hadamard Product Arguments with zero-knowledge

Definition 16. Let $i = 1, 2$. For a given $\mathbf{ck}_i \leftarrow \mathbb{G}_i^n$, $r \leftarrow \mathbb{F}$, $H_i \leftarrow \mathbb{G}_i$, the relation $\mathcal{R}_{r,n,\mathbf{ck}_i,H_i}^{\text{Elim}}$ is defined by

$$\left(\begin{array}{l} (D_1, D_2, D'_1, D'_2 \in \mathbb{G}_T ; \\ \mathbf{a}, \mathbf{c} \in \mathbb{G}_1^n, \mathbf{b}, \mathbf{d} \in \mathbb{G}_2^n, \\ r_{D_1}, r_{D_2}, r_{D'_1}, r_{D'_2} \in \mathbb{F}) \end{array} \middle| \begin{array}{l} D_1 = \langle \mathbf{a}, \mathbf{ck}_2 \rangle + r_{D_1} \cdot e(H_1, H_2), \\ D_2 = \langle \mathbf{ck}_1, \mathbf{b} \rangle + r_{D_2} \cdot e(H_1, H_2), \\ D'_1 = \langle \mathbf{c}, \mathbf{ck}_2 \rangle + r_{D'_1} \cdot e(H_1, H_2), \\ D'_2 = \langle \mathbf{ck}_1, \mathbf{d} \rangle + r_{D'_2} \cdot e(H_1, H_2) \\ \mathbf{a} \circ \mathbf{b} = \mathbf{c} \circ \mathbf{d} \end{array} \right)$$

D.2 Reformatting for Hadamard Product Arguments

Algorithm 14 Σ_{ref} with zero-knowledge

Input : $(D_1, D_2, D'_1, D'_2 ; \mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d})$
 $\mathcal{P} : r_C, r_X, r_K, r_{D_{t \in [4]}}, r_{C'}, r_{K'}, r_{D'_{t \in [4]}} \leftarrow \mathbb{F}$
 $\mathcal{P} \rightarrow \mathcal{V} : C = \langle \mathbf{a}, \mathbf{b} \rangle + r_C \cdot e(H_1, H_2), \quad C' = \langle \mathbf{c}, \mathbf{d} \rangle + r_{C'} \cdot e(H_1, H_2)$
 $\mathcal{V} \rightarrow \mathcal{P} : r \leftarrow \mathbb{F}$
 $\mathcal{P} : \mathbf{a}_r \leftarrow (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{a}, \quad \mathbf{k} \leftarrow \mathbf{1} \in \mathbb{G}_1^n,$
 $\mathbf{c}_r \leftarrow (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{c}, \quad \mathbf{k}' \leftarrow \mathbf{1} \in \mathbb{G}_1^n$
 $\mathcal{P} \rightarrow \mathcal{V} : X = \langle \mathbf{a}_r, \mathbf{b} \rangle + r_X \cdot e(H_1, H_2), \quad K = \langle \mathbf{k}, \mathbf{b} \rangle + r_K \cdot e(H_1, H_2),$
 $D_3 = \langle \mathbf{a}_r, \mathbf{ck}_2 \rangle + r_{D_3} \cdot e(H_1, H_2), \quad D_4 = \langle \mathbf{k}, \mathbf{ck}_2 \rangle + r_{D_4} \cdot e(H_1, H_2),$
 $X' = \langle \mathbf{c}_r, \mathbf{d} \rangle + r_X \cdot e(H_1, H_2), \quad K' = \langle \mathbf{k}', \mathbf{d} \rangle + r_{K'} \cdot e(H_1, H_2),$
 $D'_3 = \langle \mathbf{c}_r, \mathbf{ck}_2 \rangle + r_{D'_3} \cdot e(H_1, H_2), \quad D'_4 = \langle \mathbf{k}', \mathbf{ck}_2 \rangle + r_{D'_4} \cdot e(H_1, H_2)$
Output : $(C, X, K, D_{t \in [4]}; \mathbf{a}, \mathbf{b}, \mathbf{a}_r, \mathbf{k}, r_C, r_X, r_K, r_{D_{t \in [4]}})$
and $(C', X', K', D'_{t \in [4]}; \mathbf{c}, \mathbf{d}, \mathbf{c}_r, \mathbf{k}', r_{C'}, r_X, r_{K'}, r_{D'_{t \in [4]}})$

D.3 Relation for Elimination Folding with zero-knowledge

Definition 17. Let $i = 1, 2$. For a given $\mathbf{ck}_i \leftarrow \mathbb{G}_i^n$, $r \leftarrow \mathbb{F}$, $H_i \leftarrow \mathbb{G}_i$, the relation $\mathcal{R}_{r,n,\mathbf{ck}_i,H_i}^{\text{Elim}}$ is defined by

$$\left(\begin{array}{l} (C, X, K, D_{t \in [4]} \in \mathbb{G}_T ; \\ \mathbf{a}, \mathbf{a}_r, \mathbf{k} \in \mathbb{G}_1^n, \mathbf{b} \in \mathbb{G}_2^n, \\ r_C, r_X, r_K, r_{D_{t \in [4]}} \in \mathbb{F}) \end{array} \middle| \begin{array}{l} \mathbf{a}_r = (1, r, r^2, \dots, r^{n-1}) \circ \mathbf{a}, \\ C = \langle \mathbf{a}, \mathbf{b} \rangle + r_C \cdot e(H_1, H_2), \\ X = \langle \mathbf{a}_r, \mathbf{b} \rangle + r_X \cdot e(H_1, H_2), \\ K = \langle \mathbf{k}, \mathbf{b} \rangle + r_K \cdot e(H_1, H_2), \\ D_1 = \langle \mathbf{a}, \mathbf{ck}_2 \rangle + r_{D_1} \cdot e(H_1, H_2), \\ D_2 = \langle \mathbf{ck}_1, \mathbf{b} \rangle + r_{D_2} \cdot e(H_1, H_2), \\ D_3 = \langle \mathbf{a}_r, \mathbf{ck}_2 \rangle + r_{D_3} \cdot e(H_1, H_2), \\ D_4 = \langle \mathbf{k}, \mathbf{ck}_2 \rangle + r_{D_4} \cdot e(H_1, H_2) \end{array} \right)$$

D.4 Elimination Folding Algorithm with zero-knowledge

Algorithm 15 Σ_{elim} with zero-knowledge

Input : $(C, X, K, D_{t \in [4]}; \mathbf{a}, \mathbf{b}, \mathbf{a}_r, \mathbf{k}, r_C, r_X, r_K, r_{D_{t \in [4]}})$
Precompute : $H = e(H_1, H_2)$, $\chi = \langle \mathbf{ck}_1, \mathbf{ck}_2 \rangle$
 $\Delta_{1L} = \langle \mathbf{ck}_{1L}, \mathbf{ck}_2 \rangle$, $\Delta_{2L} = \langle \mathbf{ck}'_1, \mathbf{ck}_{2L} \rangle$
 $\Delta_{1R} = \langle \mathbf{ck}_{1R}, \mathbf{ck}_2 \rangle$, $\Delta_{2R} = \langle \mathbf{ck}'_1, \mathbf{ck}_{2R} \rangle$
 $\mathcal{P} : r_{X_L}, r_{X_R}, r_{D_{3L}}, r_{D_{3R}}, r_{D'_{1L}}, r_{D'_{1R}}, r_{D'_{2L}}, r_{D'_{2R}}, r_{D'_{3L}}, r_{D'_{3R}}, r_{D'_{4L}}, r_{D'_{4R}} \leftarrow \mathbb{F}$
 $\mathcal{P} \rightarrow \mathcal{V} : X_L = \langle \mathbf{a}_{rL}, \mathbf{b}_L \rangle + r_{X_L} H$, $X_R = \langle \mathbf{a}_{rR}, \mathbf{b}_R \rangle + r_{X_R} H$
 $D_{3L} = \langle \mathbf{a}_{rL}, \mathbf{ck}_{2L} \rangle + r_{D_{3L}} H$, $D_{3R} = \langle \mathbf{a}_{rR}, \mathbf{ck}_{2R} \rangle + r_{D_{3R}} H$
 $D'_{1L} = \langle \mathbf{a}_L, \mathbf{ck}'_2 \rangle + r_{D'_{1L}} H$, $D'_{1R} = \langle \mathbf{a}_R, \mathbf{ck}'_2 \rangle + r_{D'_{1R}} H$
 $D'_{2L} = \langle \mathbf{ck}'_1, \mathbf{b}_L \rangle + r_{D'_{2L}} H$, $D'_{2R} = \langle \mathbf{ck}'_1, \mathbf{b}_R \rangle + r_{D'_{2R}} H$
 $D'_{3L} = \langle \mathbf{a}_{rL}, \mathbf{ck}'_2 \rangle + r_{D'_{3L}} H$, $D'_{3R} = \langle \mathbf{a}_{rR}, \mathbf{ck}'_2 \rangle + r_{D'_{3R}} H$
 $D'_{4L} = \langle \mathbf{k}_L, \mathbf{ck}'_2 \rangle + r_{D'_{4L}} H$, $D'_{4R} = \langle \mathbf{k}_R, \mathbf{ck}'_2 \rangle + r_{D'_{4R}} H$
 $\mathcal{V} \rightarrow \mathcal{P} : \beta \leftarrow \mathbb{F}$
 $\mathcal{P} : \mathbf{b}^* \leftarrow \mathbf{b} + \beta^{-1} \mathbf{ck}_2$, $\mathbf{k}^* \leftarrow \mathbf{k} + \beta \mathbf{ck}_1$, $r_K^* \leftarrow r_K + \beta r_{D_2} + \beta^{-1} r_{D_4}$
 $\mathcal{P} : r_{C_{\pm}}, r_{X_{\pm}}, r_{K_{\pm}} \leftarrow \mathbb{F}$
 $\mathcal{P} \rightarrow \mathcal{V} : C_+ = \langle \mathbf{a}_L, \mathbf{b}_R^* \rangle + r_{C_+} H$, $C_- = \langle \mathbf{a}_R, \mathbf{b}_L^* \rangle + r_{C_-} H$
 $X_+ = \langle \mathbf{a}_{rL}, \mathbf{b}_R^* \rangle + r_{X_+} H$, $X_- = \langle \mathbf{a}_{rR}, \mathbf{b}_L^* \rangle + r_{X_-} H$
 $K_+ = \langle \mathbf{k}_L^*, \mathbf{b}_R^* \rangle + r_{K_+} H$, $K_- = \langle \mathbf{k}_R^*, \mathbf{b}_L^* \rangle + r_{K_-} H$
 $\mathcal{V} \rightarrow \mathcal{P} : \alpha \leftarrow \mathbb{F}$
 $\mathcal{P} : \mathbf{a}' \leftarrow \alpha \mathbf{a}_L + \mathbf{a}_R$, $\mathbf{b}' \leftarrow \alpha^{-1} \mathbf{b}_L^* + \mathbf{b}_R^*$
 $\mathbf{a}'_r \leftarrow \alpha \mathbf{a}_{rL} + r^{-\frac{n}{2}} \mathbf{a}_{rR}$, $\mathbf{k}' \leftarrow \alpha \mathbf{k}_L^* + \mathbf{k}_R^*$
 $r_{C'} \leftarrow r_C + \beta^{-1} r_{D_1} + \alpha r_{C_+} + \alpha^{-1} r_{C_-}$
 $r_{X'} \leftarrow r_{X_L} + r^{-\frac{n}{2}} r_{X_R} + \beta^{-1} (r_{D_{3L}} + r^{-\frac{n}{2}} r_{D_{3R}}) + \alpha r_{X_+} + \alpha^{-1} r^{-\frac{n}{2}} r_{X_-}$
 $r_{K'} \leftarrow r_K + \alpha r_{K_+} + \alpha^{-1} r_{K_-}$
 $r_{D'_1} \leftarrow \alpha r_{D'_{1L}} + r_{D'_{1R}}$, $r_{D'_2} \leftarrow \alpha^{-1} r_{D'_{2L}} + r_{D'_{2R}}$
 $r_{D'_3} \leftarrow \alpha r_{D'_{3L}} + r^{-\frac{n}{2}} r_{D'_{3R}}$, $r_{D'_4} \leftarrow \alpha^{-1} r_{D'_{4L}} + r_{D'_{4R}}$
 $\mathcal{V} : X \stackrel{?}{=} X_L + X_R$, $D_3 \stackrel{?}{=} D_{3L} + D_{3R}$
 $C' \leftarrow C + \beta^{-1} D_1 + \alpha C_+ + \alpha^{-1} C_-$
 $X' \leftarrow X_L + r^{-\frac{n}{2}} X_R + \beta^{-1} (D_{3L} + r^{-\frac{n}{2}} D_{3R}) + \alpha X_+ + \alpha^{-1} r^{-\frac{n}{2}} X_-$
 $K' \leftarrow K + \chi + \beta D_2 + \beta^{-1} D_4 + \alpha K_+ + \alpha^{-1} K_-$
 $D'_1 \leftarrow \alpha D'_{1L} + D'_{1R}$
 $D'_2 \leftarrow \alpha^{-1} D'_{2L} + D'_{2R} + \alpha^{-1} \beta^{-1} \Delta_{2L} + \beta^{-1} \Delta_{2R}$
 $D'_3 \leftarrow \alpha D'_{3L} + r^{-\frac{n}{2}} D'_{3R}$
 $D'_4 \leftarrow \alpha D'_{4L} + D'_{4R} + \alpha \beta \Delta_{1L} + \beta \Delta_{1R}$
Output : $(C', X', K', D'_{t \in [4]}; \mathbf{a}', \mathbf{b}', \mathbf{a}'_r, \mathbf{k}', r_{C'}, r_{X'}, r_{K'}, r_{D'_{t \in [4]}})$

Theorem 6. Let $i = 1, 2$. For $\mathbf{ck}'_i \leftarrow \mathbb{G}_i^{n/2}$, $H_i \leftarrow \mathbb{G}_i$, Σ_{elim} is a HVSZK, public-coin, succinct interactive argument of knowledge for $\mathcal{R}_{r, n/2, \mathbf{ck}_1, \mathbf{ck}_2, H_i}^{\text{Elim}}$ with $(9, 12/|\mathbb{F}|)$ -tree extractability under SXDH.

Proof of HVSZK. The validity of HVSZK holds since all messages except X_L , X_R , D_{3L} , and D_{3R} from $\mathcal{P}$ to $\mathcal{V}$ consist of uniformly random elements of $\mathbb{G}_T$, and thus can be trivially simulated. The variables X_L , X_R , D_{3L} , and D_{3R} can also be simulated by initially sampling X_L and D_{3L} from $\mathbb{G}_T$, followed by computing $X_R \leftarrow X - X_L$ and $D_{3R} \leftarrow D_3 - D_{3L}$. $\square$

D.5 Pairing Check Algorithm with zero-knowledge

Algorithm 16 Σ_{check} with zero-knowledge

Input : $(C, X, K, D_{t \in [4]}; a, b, a_r, k, r_C, r_X, r_K, r_{D_{t \in [4]}})$
Precompute : $H = e(H_1, H_2)$, $\chi = e(ck_1, ck_2)$
 $\mathcal{P} : r_{P_1}, r_{P_2}, r_{P_3}, r_{Q_1}, r_{Q_2}, r_{Q_3}, r_R \leftarrow \mathbb{F}$,
 $d_1 \leftarrow \mathbb{G}_1, d_2 \leftarrow \mathbb{G}_2$
 $\mathcal{P} \rightarrow \mathcal{V} : P_1 = e(d_1, ck_2) + r_{P_1}H, \quad P_2 = e(ck_1, d_2) + r_{P_2}H, \quad P_3 = e(k, d_2) + r_{P_3}H,$
 $Q_1 = e(d_1, b) + e(a, d_2) + r_{Q_1}H, \quad Q_2 = e(a, d_2) + r_{Q_2}H,$
 $Q_3 = e(a_r, d_2) + r_{Q_3}H, \quad R = e(d_1, d_2) + r_RH$
 $\mathcal{V} \rightarrow \mathcal{P} : c \leftarrow \mathbb{F}$
 $\mathcal{P} \rightarrow \mathcal{V} : E_1 \leftarrow d_1 + ca + c^3a + c^5a_r + c^7k,$
 $E_2 \leftarrow d_2 + cb,$
 $r_1 \leftarrow cr_{D_1} + c^3(1 + c^2)r_{D_3} + c^7r_{D_4} + r_{P_1}, \quad r_2 \leftarrow cr_{D_2} + r_{P_2},$
 $r_3 \leftarrow c^2r_C + c^4(1 + c^2)r_X + c^8r_K + c(r_{Q_1} + c^2r_{Q_2} + c^4r_{Q_3} + c^6r_{P_3}) + r_R$
 $\mathcal{V} : d \leftarrow \mathbb{F}$, accept if :
 $e(E_1 + d \cdot ck_1, E_2 + d^{-1} \cdot ck_2)$
 $= \chi + c^2C + c^4(1 + c^2)X + c^8K + cd^{-1}D_1 + c^3d^{-1}(1 + c^2)D_3 + c^7d^{-1}D_4$
 $+ c(Q_1 + c^2Q_2 + c^4Q_3 + c^6P_3) + d^{-1}P_1 + cdD_2 + dP_2 + R - (d^{-1}r_1 + dr_2 + r_3)H$

Theorem 7. Let $i = 1, 2$. For $ck_i \leftarrow \mathbb{G}_i$, $H_i \leftarrow \mathbb{G}_i$, Σ_{check} is a HVSZK, public-coin, succinct interactive argument of knowledge for $\mathcal{R}_{r,1,ck_1,ck_2,H_1,H_2}^{\text{Elim}}$ with $(27, 27/|\mathbb{F}|)$ -tree extractability under SXDH.

Proof of HVSZK. To construct a simulator, the following steps are undertaken:

1. Sample $Q_1, Q_2, Q_3, E_1, E_2, P_3$ from $\mathbb{G}_T$.
2. Compute the challenge c using $\mathcal{V}$'s coins.
3. Sample r_1, r_2, r_3 from $\mathbb{F}$.
4. Compute P_1, P_2, R as follows: $P_1 = e(E_1, ck_2) - c(D_1 + c^2D_3 + c^4D_3 + c^6D_4)$,
 $P_2 = e(ck_1, E_2) - cD_2$, and $R = e(E_1, E_2) - c(Q_1 + c^2Q_2 + c^4Q_3 + c^6P_3) -$
 $c^2(C + c^2X + c^4X + c^6K)$. $\square$

D.6 Batching Scheme with zero-knowledge

Theorem 8. Let $i = 1, 2$. For $\mathbf{ck}_i \leftarrow \mathbb{G}_i^n$, $H_i \leftarrow \mathbb{G}_i$, Batch- Σ_{elim} is a HVSZK, public-coin, succinct interactive argument of knowledge for $(\mathcal{R}_{r,n,\mathbf{ck}_1,\mathbf{ck}_2,H_i}^{\text{Elim}})^2$ with $(3, 3/|\mathbb{F}|)$ -tree extractability under SXDH.

Proof. Since Succinctness, the Public Coin property, Completeness, Soundness and HVSZK of this protocol are immediate, It is enough to show tree extractability. We have $\mu = 1$ and set $w_1 = 3$. We are given witnesses for 3 distinct challenges δ . We interpolate $\mathbf{a}$, $\mathbf{b}$, $\mathbf{a}_r$, and $\mathbf{k}$ as quadratics in δ . Then from Lemma 6 in [22], the contribution of the quadratic terms to $D_{t \in [4]} = \delta D_{1,t} + D_{2,t}$ is identically zero, and so from Lemma 3 in [22] there are no quadratic terms. Hence

Algorithm 17 Batch- Σ_{elim}

Input : $(C_i, X_i, K_i, D_{i,t \in [4]}; \mathbf{a}_i, \mathbf{b}_i, \mathbf{a}_{\mathbf{r}i}, \mathbf{k}_i, r_{C_i}, r_{X_i}, r_{K_i}, r_{D_{i,t \in [4]}})_{i \in [2]}$
Precompute : $H = e(H_1, H_2)$
 $\mathcal{P} : r_{Z_C}, r_{Z_X}, r_{Z_K} \leftarrow \mathbb{F}$
 $\mathcal{P} \rightarrow \mathcal{V} : Z_C = \langle \mathbf{a}_1, \mathbf{b}_2 \rangle + \langle \mathbf{a}_2, \mathbf{b}_1 \rangle + r_{Z_C} H,$
 $\quad Z_X = \langle \mathbf{a}_{\mathbf{r}1}, \mathbf{b}_2 \rangle + \langle \mathbf{a}_{\mathbf{r}2}, \mathbf{b}_1 \rangle + r_{Z_X} H,$
 $\quad Z_K = \langle \mathbf{k}_1, \mathbf{b}_2 \rangle + \langle \mathbf{k}_2, \mathbf{b}_1 \rangle + r_{Z_K} H$
 $\mathcal{V} \rightarrow \mathcal{P} : \delta \leftarrow \mathbb{F}$
 $\mathcal{P} : \mathbf{a} \leftarrow \delta \mathbf{a}_1 + \mathbf{a}_2, \mathbf{b} \leftarrow \delta \mathbf{b}_1 + \mathbf{b}_2,$
 $\quad \mathbf{a}_{\mathbf{r}} \leftarrow \delta \mathbf{a}_{\mathbf{r}1} + \mathbf{a}_{\mathbf{r}2}, \mathbf{k} \leftarrow \delta \mathbf{k}_1 + \mathbf{k}_2$
 $\quad r_C \leftarrow \delta^2 r_{C1} + \delta r_{Z_C} + r_{C2},$
 $\quad r_X \leftarrow \delta^2 r_{X1} + \delta r_{Z_X} + r_{X2},$
 $\quad r_K \leftarrow \delta^2 r_{K1} + \delta r_{Z_K} + r_{K2}$
 $\quad r_{D_{t \in [4]}} \leftarrow \delta r_{D_{1,t}} + r_{D_{2,t}}$
 $\mathcal{V} : C \leftarrow \delta^2 C_1 + \delta Z_C + C_2,$
 $\quad X \leftarrow \delta^2 X_1 + \delta Z_X + X_2,$
 $\quad K \leftarrow \delta^2 K_1 + \delta Z_K + K_2,$
 $\quad D_{t \in [4]} \leftarrow \delta D_{1,t} + D_{2,t}$
Output : $(C, X, K, D_{t \in [4]}; \mathbf{a}, \mathbf{b}, \mathbf{a}_{\mathbf{r}}, \mathbf{k}, r_C, r_X, r_K, r_{D_{t \in [4]}})$

$\mathbf{a} = \delta \mathbf{a}_1 + \mathbf{a}_2$, $\mathbf{b} = \delta \mathbf{b}_1 + \mathbf{b}_2$, $\mathbf{a}_{\mathbf{r}} = \delta \mathbf{a}_{\mathbf{r}1} + \mathbf{a}_{\mathbf{r}2}$ and $\mathbf{k} = \delta \mathbf{k}_1 + \mathbf{k}_2$ for some $\mathbf{a}_i$, $\mathbf{b}_i$, $\mathbf{a}_{\mathbf{r}i}$, and $\mathbf{k}_i$. We define the following interpolations:

$$\begin{aligned} r_C(\delta) &= r_{C2} + \delta r_{Z_C} + \delta^2 r_{C1}, \\ r_X(\delta) &= r_{X2} + \delta r_{Z_X} + \delta^2 r_{X1}, \\ r_K(\delta) &= r_{K2} + \delta r_{Z_K} + \delta^2 r_{K1} \end{aligned}$$

Substituting in $\mathbf{a}$, $\mathbf{b}$, $\mathbf{a}_{\mathbf{r}}$, $\mathbf{k}$:

$$\begin{aligned} C(\delta) &= \delta^2 C_1 + \delta Z_C + C_2 = \langle \mathbf{a}(\delta), \mathbf{b}(\delta) \rangle + r_C(\delta) H \\ &= \delta^2 [\langle \mathbf{a}_1, \mathbf{b}_1 \rangle + r_{C1} H] + \delta [\langle \mathbf{a}_1, \mathbf{b}_2 \rangle + \langle \mathbf{a}_2, \mathbf{b}_1 \rangle + r_{Z_C} H] + [\langle \mathbf{a}_2, \mathbf{b}_2 \rangle + r_{C2} H], \\ X(\delta) &= \delta^2 X_1 + \delta Z_X + X_2 = \langle \mathbf{a}_{\mathbf{r}}(\delta), \mathbf{b}(\delta) \rangle + r_X(\delta) H \\ &= \delta^2 [\langle \mathbf{a}_{\mathbf{r}}, \mathbf{b} \rangle + r_{X1} H] + \delta [\langle \mathbf{a}_{\mathbf{r}1}, \mathbf{b}_2 \rangle + \langle \mathbf{a}_{\mathbf{r}2}, \mathbf{b}_1 \rangle + r_{Z_X} H] + [\langle \mathbf{a}_{\mathbf{r}2}, \mathbf{b}_2 \rangle + r_{X2} H], \\ K(\delta) &= \delta^2 K_1 + \delta Z_K + K_2 = \langle \mathbf{k}(\delta), \mathbf{b}(\delta) \rangle + r_K(\delta) H \\ &= \delta^2 [\langle \mathbf{k}_1, \mathbf{b}_2 \rangle + r_{K1} H] + \delta [\langle \mathbf{k}_1, \mathbf{b}_2 \rangle + \langle \mathbf{k}_2, \mathbf{b}_1 \rangle + r_{Z_K} H] + [\langle \mathbf{k}_2, \mathbf{b}_2 \rangle + r_{K2} H]. \end{aligned}$$

As this holds for three values of δ , Lemma 6 in [22] implies that the two polynomials have identical coefficients, yielding:

$$\begin{aligned} C_i &= \langle \mathbf{a}_i, \mathbf{b}_i \rangle + r_{C_i} H, \\ X_i &= \langle \mathbf{a}_{\mathbf{r}i}, \mathbf{b}_i \rangle + r_{X_i} H, \\ K_i &= \langle \mathbf{k}_i, \mathbf{b}_i \rangle + r_{K_i} H \end{aligned}$$

Since $\mathbf{a}_{\mathbf{r}i} = \mathbf{r} \circ \mathbf{a}_i$, we deduce:

$$\mathbf{a}_{\mathbf{r}} = \delta \mathbf{a}_{\mathbf{r}1} + \mathbf{a}_{\mathbf{r}2} = \delta \mathbf{r} \circ \mathbf{a}_1 + \mathbf{r} \circ \mathbf{a}_2 = \mathbf{r} \circ (\delta \mathbf{a}_1 + \mathbf{a}_2) = \mathbf{r} \circ \mathbf{a}.$$

Thus, we have extracted the required witnesses. $\square$